

A LOW-COST 3D-MAPPING SOLUTION USING A 2D LIDAR MOUNTED ON A DRONE

JAN LESZEK GÓRSKI 4132830

Master's Project in Engineering (Robot Systems)
Supervisor: Ph.D. Elżbieta Pastucha

June 2026



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Word count: 12241

Abstract

3D LiDAR (*Light Detection And Ranging*) sensors are widely used in modern vehicles for navigation and mapping tasks. However, while their price got lower in recent years, this technology is still not at a consumer-grade level. In contrast, 2D LiDAR modules are already a robust and widely available solution. In this project, the author designs and evaluates a quadcopter drone equipped with a companion computer and a low-cost 2D LiDAR for mapping and autonomous navigation. The main goal is to explore the limitations of the cheaper sensor, and if it can still be used reliably in these tasks. GNSS (*Global Navigation Satellite System*), optical flow, and OptiTrack positioning systems are compared to find their influence on the quality of the obtained scans. Despite introducing filtration methods, like ICP (*Iterative Closest Point*) or OctoMap, their performance is not satisfactory for high-precision mapping. However, the gathered point clouds can still be successfully used for the obstacle avoidance by utilizing the EGO-Swarm trajectory planner software. The code used in this project is released as an open-source package on a public repository.

Contents

Contents	II
1 Introduction	1
2 Background	3
3 Analysis and Design	5
3.1 Components	5
3.1.1 2D LiDAR Module	5
3.1.2 Companion Computer	6
3.1.3 Drone Platform	7
3.2 Assembly	10
4 Implementation	14
4.1 ROS 2 Nodes	14
4.2 Simulation	15
4.3 Data Acquisition	17
4.3.1 LiDAR Scans	17
4.3.2 Drone Pose	19
4.3.3 Scan Stacking	21
4.4 Filtration	23
4.4.1 Error Sources	23
4.4.2 Filtration Methods	26
4.5 Mapping	30
4.6 Navigation	34
5 Results	37
5.1 Mapping	37
5.1.1 LiDAR Accuracy Test	37
5.1.2 Positioning Accuracy Test	39
5.1.3 Point Clouds	42
5.1.4 Summary	51
5.2 Navigation	52
5.3 Cost Estimate	55
6 Conclusions	57
6.1 Future Work	58

A Appendix	59
Bibliography	60

1 Introduction

Thanks to recent advancements in the drone technology, autonomous operations are becoming the standard use of unmanned vehicles. In order to achieve this, the vehicles require a perception of their surroundings. Typical drones utilize cameras or depth sensors, which, combined with a memory of already visited places, enable the construction of expanding maps used for autonomous navigation and localization. Furthermore, these mapping systems can also be used for digitalizing large-scale or inaccessible objects, like, for example, decorative sculptures or other distinctive features of buildings or landmarks.

LiDAR (*Light Detection And Ranging*) sensors mounted on drones enable them to scan and digitize various objects or environments as 3D point clouds. They emit infrared laser beams, which can be reflected from the encountered obstacles back to sensor. By measuring the time between the emission and the return detection, an accurate distance can be calculated. However, the full 3D LiDAR systems are still either expensive or have too narrow field of view ($\sim 2^\circ$) to be used effectively. In contrast, the 2D LiDAR modules are becoming increasingly accessible to everyday users with their low price ($\sim \text{€}60$). With the addition of a positioning system, like GNSS (*Global Navigation Satellite System*) coupled with the IMU (*Inertial Measurement Unit*) orientation data, their 2D readings can be expanded into the 3D space.

To address this issue, the author proposes a budget-friendly system made of widely used components in the hobby drone community. By adding an inexpensive 2D LiDAR sensor to an entry-level FPV (*First-Person View*) freestyle drone, and combining it with a single-board companion computer, it is possible to build an affordable mapping system. The LiDAR sensor was mounted directly on the drone, to explore the limitations of this particular design. Compared to other solutions, often based on cameras or rotating 2D LiDARs, the proposed system should be also less computationally expensive, ideally working in real-time regimes. Its tradeoffs, however, are less dense point clouds and necessity of a drone movement to accumulate 3D scans.

The overall goal of this project is to gather 2D LiDAR readings and, knowing drone's pose in a given time, create a 3D map of its surroundings. Due to the limited positioning data accuracy, filtration and correction methods are proposed. The performance of the system was checked, using a simple GNSS antenna, an optical flow sensor, and the OptiTrack Motion Capture System [1] located in the SDU UAS Test Center [2]. The last one serves a superior position sensor to simulate the use of high-precision systems, like an RTK (*Real-Time Kinematic*). Finally, the proposed system was evaluated for the 3D mapping and obstacle-avoidance navigation tasks. The software created for this project is based on the ROS 2 (*Robot Operating System*) [3] framework and follows the open-source standards.

The proposed system consists of the LDROBOT D500 2D LiDAR sensor [4] mounted on top of the GEPRC Mark4 drone [5]. The drone runs the open-source ArduCopter firmware [6] on the JJIEFPV H743 flight controller [7]. The obtained data is processed on the Orange Pi Zero 2W companion computer [8]. The components were chosen based on their affordability to keep the overall cost as low as possible.

2 Background

The LiDAR-based mapping and navigation have become increasingly popular across mobile platforms in recent years. The most significant advantage of these sensors over traditional cameras is that they can work even in poor lighting conditions. However, their performance may worsen in rainy or densely fogged environments, as water particles disrupt the laser beams [9].

Several researchers experimented with LiDAR equipped UAVs (*Unmanned Aerial Vehicles*). Wang et al. [10] described a loop-closure detection problem for a drone fitted with a 2D laser rangefinder. Sequential laser scanning data was aligned into a 3D point cloud map, using the pose information provided by the IMU, GNSS, and optical flow sensors. This rough map was then reconstructed, using a 2D scan matching method proposed by Thrun et al. [11] to mitigate the positioning errors. Then, the authors used an incremental sliding window algorithm to extract and match the key features in a current scene. In the result, the drone was capable of creating 3D maps and recognizing the already visited places.

The concept of creating 3D maps with 2D LiDARs was explored by Bi et al. [12]. The authors presented various mounting configurations and compared them to highlight both their strengths and weaknesses. Typical problems of moving 2D LiDAR platforms, like errors in the position estimation, real-time performance issues, or 3D SLAM (*Simultaneous Localization And Mapping*) implementations, were also described.

Kumar et al. [13] integrated 2 orthogonally-mounted 2D LiDARs and an IMU sensor to create a navigation system for UAVs. They used the point-to-point matching-based method — PSM (*Polar Scan Matching*) [14] — to determine the translation of LiDAR scans during the flight. This approach is known for its low computational cost, ideal for small embedded devices. A Kalman filter was used to combine the 2D position from the scan matching with the altitude information. The proposed method produced detailed maps during indoor flights, without a need for a GNSS antenna, and successfully supported a pipeline classification algorithm.

An example of a LiDAR-based navigation was presented by Hou et al. [15]. Their algorithm consists of map construction, target selection and motion planning. To enhance the map generation, 2D LiDAR scans were fused with depth camera images, which allowed for the majority of the outlying point clusters to be filtered out. The proposed navigation algorithm was compared with FUEL (*Fast UAV ExpLoration*) [16] and FAEP (*Fast Autonomous Exploration Planner*) [17] algorithms, and it was found that the exploration time was reduced by 19% and 13% respectively.

In the paper by Faria et al. [18], a 2D-LiDAR-equipped UAV was able to create 3D maps by combining the frontier-based exploration [19], the Lazy Theta* path planner [20], and flyby sampling manoeuvres. Positioning was assumed to be handled

by a differential GNSS (outdoors), and a VICON/OptiTrack system (indoors), as the main goal was to develop the exploration algorithm itself. The resulting solution was capable of autonomous exploration with almost full volume coverage of a defined space.

Zhang et al. [21] developed a real-time method for the 2D-LiDAR-based odometry and 3D mapping, in which the position of the sensor could be established, using feature detection algorithms. Distinctive attributes were extracted from sharp edges and planar surface patches on individual scans with co-planar geometric relationship. Testing both in the simulation and in the real world produced great results, with the average position error of 0.88% of the distance travelled.

Based on this work, Wen et al. [22] proposed a framework for the cooperative semantic labelling, map generation, and line model extraction to improve their respective accuracies, using LiDAR data. A dedicated neural network extracted the semantic information from rough local maps, which was then used to make the plane detection more efficient and optimize the system's pose estimation.

Zhen and Scherer [23] proposed a unified framework for LiDAR-based SLAM. Their work was an extension of the Cartographer SLAM algorithm [24] for rotating 2D and 3D LiDARs. The combination of an ESKF (*Error State Kalman Filter*) and a GPF (*Gaussian Particle Filter*) made it possible to estimate robot states inside a prior-known map without a need for external positioning sensors.

The 3D mapping correction was also presented by Hassan et al. [25]. There, a 2D LiDAR was mounted on a crane boom. The authors used the plane fitting together with the general graph optimization (*g2o*) [26] techniques to increase the mapping precision of the scanned elements. This method proved to be accurate, but the map correction algorithm could not operate in real-time regimes.

Other implementations, like these by Liu et al. [27] and Li et al. [28], combine 2D LiDARs with RGB cameras to further enhance the mapping process. By fusing the LiDAR, image, and IMU data, these approaches produce more precise, dense colour maps, but at the expense of higher computational requirements.

3 Analysis and Design

3.1 Components

3.1.1 2D LiDAR Module

The 2D LiDAR sensor is the most crucial component in this project. Ideally, it should be lightweight, consume a low amount of energy, and be available at a low price. For the consumer-grade solutions (under €150), there are the following modules to choose from:

- RPLIDAR A1 [29],
- LDROBOT D500 [4],
- RPLIDAR C1 [30],
- LDROBOT D800 [31].

The sensors are compared in Table 3.1. In this price range, their characteristics are similar. The only exception is the LDROBOT D800, which has approximately twice the range and accuracy of the other sensors, but it is also the most expensive one. The RPLIDAR A1 has the best power efficiency, but its accuracy is low at distances over 5 m.

Model	Range [m]	Scanning Frequency [Hz]	Accuracy [m]	Anti-ambient Light [lx]	Power Consumption [W]	Price [€]
RPLIDAR A1	0.15–12	2–10	0.3	N/A	0.5	~110
LDROBOT D500	0.03–12	6–13	0.03	60000	1.45	~60
RPLIDAR C1	0.05–12	10	0.03	40000	1.15	~90
LDROBOT D800	0.03–25	10	0.015	60000	1.45	~130

Table 3.1: LiDARs comparison.

In the end, it was decided to use the LDROBOT D500 module (Figure 3.1). While it has the lowest price of all compared LiDARs, its performance is not worse. Its only downside is the higher power consumption which can lead to shorter drone's flight times. The maximal range of 12 m is acceptable in most of the scanning and navigation tasks. It is also resilient enough to sunlight, which makes it capable of

working in outdoor environments. The module uses the UART (*Universal Asynchronous Receiver-Transmitter*) serial communication to transmit data. Additionally, the manufacturer provides an SDK (*Software Development Kit*) for ROS 2 [32][33], which cuts down the development time.

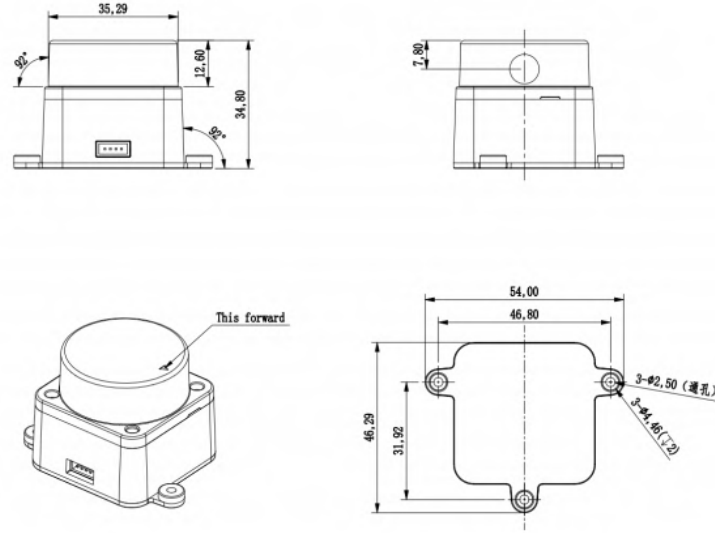


Figure 3.1: LDROBOT D500 schematics [34].

3.1.2 Companion Computer

The companion computer should be as small as possible, but, in the same time, have enough computing power to run ROS 2 and manipulate point clouds. Its power consumption is also an important factor, as it limits the drone's flight time. The following small-factor, single board computers were compared:

- Radxa ZERO 3W [35],
- Orange Pi Zero 2W [8],
- Raspberry Pi Zero 2 W [36].

The results are presented in Table 3.2. The most power-efficient board is the Raspberry Pi Zero 2 W. It comes with the cost of the least powerful components among the compared computers. With 8 GB of RAM and USB-C 3.0, the Radxa ZERO 3W seems to be the most suitable option, but its power consumption is significantly larger. Its price is also much higher.

Finally, the Orange Pi Zero 2W was chosen for this project (Figure 3.2). It is an optimal compromise between the computing power and the price. With 4 GB of RAM, it is able to run Ubuntu 24.04 Server [37] with ROS 2 Jazzy [38]. Dedicated ports enable fast communication with peripherals, which is crucial for time synchronization

Model	CPU	RAM	Connectivity	Wi-Fi	Power Consumption [W]	Price [€]
Radxa ZERO 3W	Quad-Core Arm Cortex-A55 64-bit 1.6 GHz + Arm Mali-G52-2EE	8 GB LPDDR4	1× USB-C 3.0 1×USB-C 2.0 40×GPIO 5×UART 1×SPI 2×I2C 1×PCM/I2S 6×PWM	Wi-Fi 6	~4–5	~90
Orange Pi Zero 2W	Allwinner H618 Quad-Core Cortex-A53 1.5 GHz + Arm Mali-G31-MP2	4 GB LPDDR4	2×USB-C 2.0 40×GPIO 5×UART 1×SPI 3×I2C 3×PWM	Wi-Fi 5	~3–4	~30
Raspberry Pi Zero 2 W	Quad-Core Arm Cortex-A53 64-bit 1 GHz	512 MB LPDDR2	2×microUSB 2.0 40×GPIO 2×UART 2×SPI 2×I2C 2×PWM	Wi-Fi 4	~2–3	~13

Table 3.2: Companion computers comparison.

between different sensors readings. To reduce the thermal throttling, 2 additional heat sinks are installed on the CPU and RAM chips, as the board has only a passive cooling system.

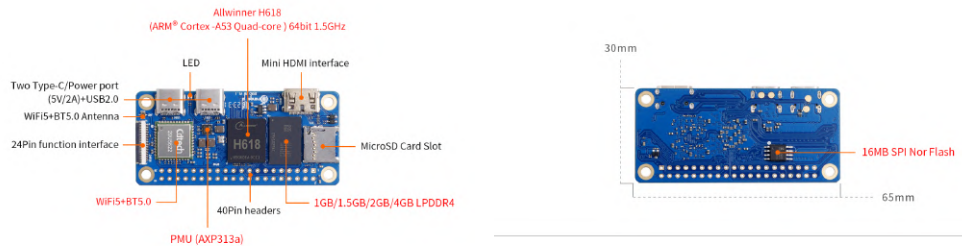


Figure 3.2: Orange Pi Zero 2W companion computer [8].

3.1.3 Drone Platform

The drone platform was chosen based on the following requirements:

- a large enough size to mount the LiDAR sensor, the companion computer, and all required sensors on the frame,

- a sufficient battery to power the LiDAR sensor, the companion computer, and the rest of the drone's components,
- accurate sensors capable of estimating the drone's position and orientation.

Frame

For the drone's base, the GEPRC Mark4 [5] was chosen (Figure 3.3). The 7-inch frame is large enough to fit the selected LDROBOT D500 LiDAR. Its sandwich design, held with 16 M3 screws, creates multiple mounting points for additional equipment. It is also convenient to attach custom 3D-printed parts to it.



Figure 3.3: GEPRC Mark4 7-inch frame [5].

Propulsion

The frame was combined with standard A2807-1300KV BLDC (*BrushLess Direct Current*) motors [39] and 7040 tri-blade propellers. Together with the JIJIEFPV BLS 70 A ESC (*Electronic Speed Controller*) [7], it is more than sufficient to drive a drone of this size. The drone is powered by the 5S 40C 2200 mAh Li-Po (*Lithium-Polymer*) battery, which should last for 10–15 minutes of the flight time.

Flight Controller

Initially, the Omnibus F4 V3 Plus [40] with the INAV firmware [41], was supposed to be used as the drone's flight controller. However, problems with DShot [42] protocol in INAV 9.0.0 version and its worse positioning accuracy, resulted in switching to another firmware. As an alternative, it was decided to use more expensive and powerful JIJIEFPV H743 flight controller [7] with the ArduCopter firmware [6]. In ArduCopter, obtaining a local position of a drone is straightforward — it is a matter of a single MAVLink command stream [43]. It is calculated internally, using an extended Kalman filter, based on available sensors. Contrary, in INAV a local position would need to be calculated manually from individual MSP (*Multiwii Serial Protocol*) [44] queries for each sensor. Then, the obtained data would need to be put into an external Kalman filter, which would result in significant delays and synchronization problems. Furthermore, the Omnibus F4 V3 Plus board does not have enough dedicated UART

ports, and could only communicate with the companion computer, using a software UART emulation. This emulation can only work with speeds up to 19 200 bit/s [45], so it would be a major transfer bottleneck.

The JIJIEFPV H743 flight controller (Figure 3.4) has the following features:

- STM32H743 32-bit 480 MHz processor [46],
- ICM42688 dual IMU [47],
- SPL06 barometer [48],
- 8 UART ports (5 DMA (*Direct Memory Access*) capable),
- 1 I2C (*Inter-Integrated Circuit*) port.

This hardware is more than enough to use all the required sensors and high-speed communication buses.

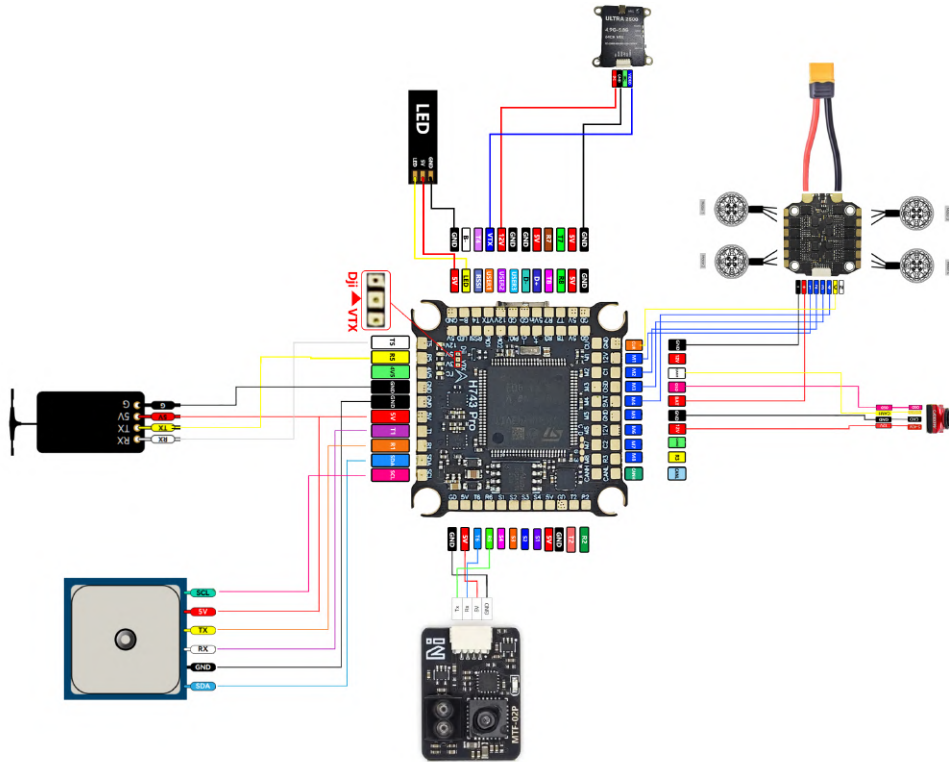


Figure 3.4: JIJIEFPV H743 flight controller wiring diagram [49][50].

Positioning

The position information is gathered by the SoloGood M10-180C GNSS antenna [51] with the additional QMC5883L magnetometer [52]. Its positioning accuracy of 1.5 m is not ideal, but it can be enhanced by the extended Kalman filtering.

Additionally, the drone is equipped with the MicoAir MTF-02P optical flow sensor [50]. In good lighting conditions, it can accomplish up to 2 cm of altitude accuracy, and work with speeds up to 7 m/s. Thanks to that, the drone is able to operate in GPS-denied environments as well.

For the OptiTrack tests, special reflective markers [53] are mounted on the drone. In theory, this system can achieve up to 0.2 mm of positional and 0.1° of rotational accuracy with typical latency of less than 10 ms.

Miscellaneous

SpeedyBee Nano 2.4 GHz ELRS [54] receiver [55] is used as a communication link on the drone. Its frequency can interrupt the Wi-Fi network of the companion computer, but, as this drone is not used in long-range scenarios, it was decided to mitigate this problem with a channel separation (described further down). Ideally, a 900 MHz receiver would be a better choice.

The drone is controller with the Jumper T-Pro S 2.4 GHz ELRS radio [56]. It is also equipped with a 200 mW, 5.8 GHz wireless video transmission (VTX) system, but it is not used in the scope of this project.

3.2 Assembly

First, the drone was modelled and assembled in the CAD (*Computer-Aided Design*) software (Figure 3.5). It helped with designing the necessary 3D-printed PLA parts and assembling the components. Additionally, the assembly model could be later used in the simulation. Furthermore, it is used to determine the exact transformation from the LiDAR's to the drone's frame of reference, eliminating the need for the LiDAR's position calibration.

The LiDAR sensor is mounted pitching down by 30° and elevated, in a way that no part of the drone obstructs its field of view. In this way, the drone is able to detect a potential floor or ceiling, without a need to change the orientation, resulting in its movement. These features allow stationary 'look-around' yaw turns to obtain new information about the drone's surrounding. The LiDAR module is connected to the frame through 3 silicon dampers to reduce the vibrations generated by the spinning motors. Without them, the in-flight readings could be potentially distorted.

The companion computer is placed between the top part of the carbon frame and the battery, in a custom-designed compartment (see Figure 3.6 for close-ups). The GNSS module is placed in the tail section, to keep the compass away from the electromagnetic interference produced by the ESC and the power cables. The optical flow sensor is mounted there as well to give it with a ground view clearance.



Figure 3.5: CAD design of the drone.

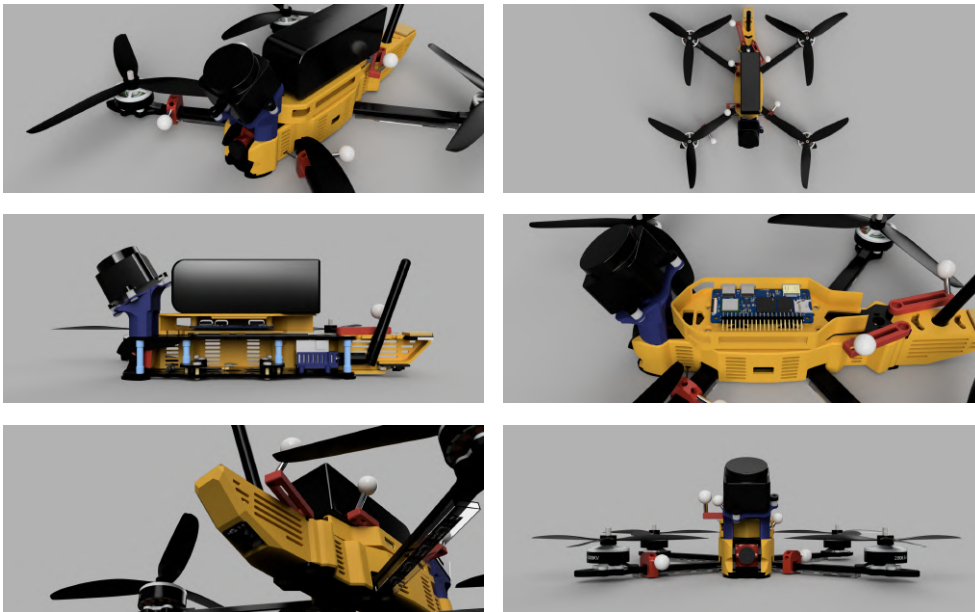


Figure 3.6: CAD design close-ups.

The system design structure is shown in Figure 3.7. The LDROBOT D500 LiDAR uses the UART communication protocol with 230 400 bit/s baud rate. Ideally, it should be connected directly to the Orange Pi's UART port, but, for the sake of a better modularity, it was decided to change it to the USB-C 2.0 port. This way, it is much more convenient to remove the LiDAR, if needed. However, this solution requires an additional FTDI USB-UART converter [57], which introduces a small processing delay. Still, as the baud rate of the communication link is high enough and the LiDAR publishes new data approximately every 100 ms, this delay is acceptable.

The JIJIEFPV H743 flight controller uses the MAVLink protocol for the telemetry link with the companion computer. They are connected together through their dedicated UART ports (the flight controller's one is DMA capable to further decrease

the transmission delay). In order to handle large data streams, the UART speed is set to 500 000 bit/s.

The positioning sensors are directly connected to and handled by the flight controller. Then, after refining with the IMU data and all necessary filtrations, the position information can be sent further. The flight controller also provides the 5 V power source for the companion computer.

To reduce the interference between radio links used by the drone, a channel separation is required. The standard 2.4 GHz Wi-Fi connection between the companion computer and a ground station would be disrupted by the 2.4 GHz ELRS receiver. To address this issue, it was decided to change the connecting network to a 5 GHz one (exactly 5.745 GHz). However, this required changing the VTX band to 5.9545 GHz as well. Without this 200 MHz frequency separation, the Wi-Fi network would be completely jammed by the video transmission.

Similarly, the OptiTrack system sends the position data to the companion computer over its 5.2 GHz Wi-Fi network. It is then captured by the ROS 2 node running on the companion computer.

Additionally, for the system's state monitoring, a single WS2812B RGB LED [58] is mounted on top of the drone's frame.

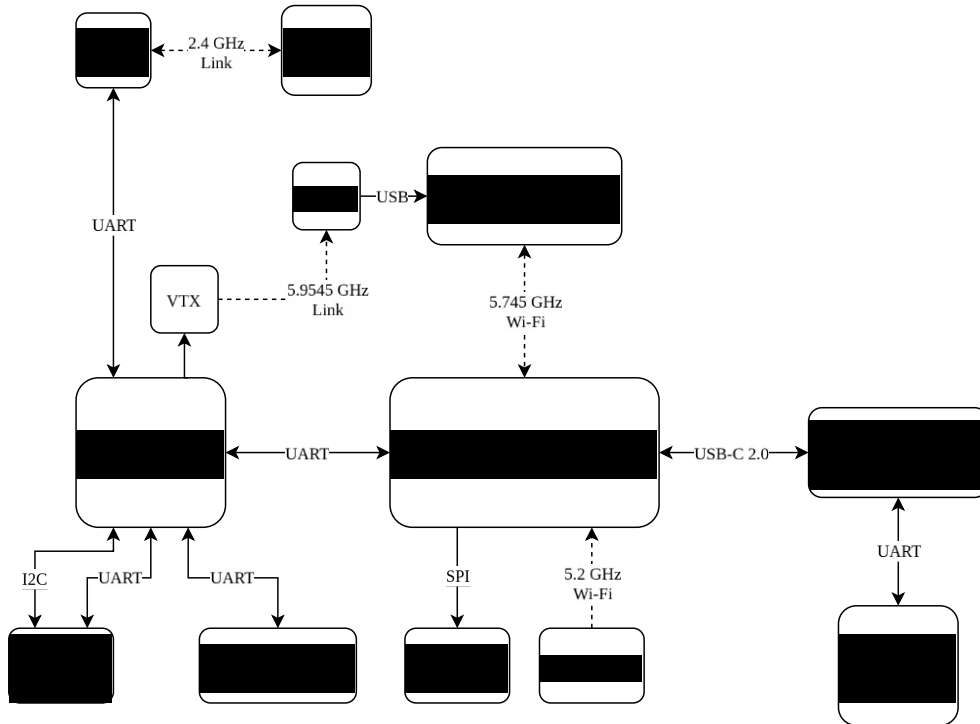


Figure 3.7: System design graph.

The assembled drone is shown in Figure 3.8. Its maximal dimensions are: 37.3 cm $\times$ 40 cm $\times$ 10.2 cm, and it weighs approximately 985 g.



Figure 3.8: Assembled drone.

4 Implementation

4.1 ROS 2 Nodes

The ROS 2 framework was chosen as the top-layer software, integrating multiple sensors readings and commands for the drone. Although it is not ideal for industrial-grade solutions, it provides major benefits for the rapid prototyping. All the required information could be recorded as topics during the drone's flight and then used as an input in the algorithm development phase. These sensor readings can be easily accessed online over the network for faster debugging. ROS 2 can be also easily integrated with simulators, like Gazebo [59]. The same software can run in the simulation and, without any modifications, on the physical platform. Together with many open-source and reliable libraries, it significantly decreases the required development time.

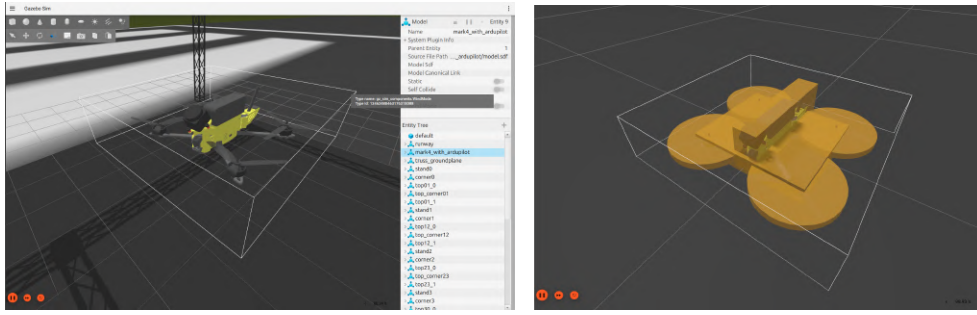
The software components are divided into smaller subprograms, called nodes. Each node is responsible for only one specific task, and they can share information between each other, using topics. There are the following nodes present:

- `lidar_mapper` — main node for the point cloud reconstruction and data integration,
- `ldlidar_node` — LiDAR communication and initial data filtration,
- `mavros_node` — MAVLink layer, handling the communication between the companion computer and the flight controller,
- `octomap_server_node` — point-cloud-based OctoMap generation,
- `ego_planner_node` — obstacle-avoidance path planner for the drone,
- `traj_server` — path trajectory server,
- `ldlidar_qos_bridge` — quality of service translation layer for the path planner,
- `mocap_relay_node` — relay for the OptiTrack positioning data,
- `lidar_mapper_visualizer` — visualization tool for the obtained point clouds.

4.2 Simulation

In the simulation, the developed algorithms can be safely tested and evaluated, without a risk of destroying the actual drone. Because of its tight integration with the ROS 2 Jazzy framework, it was decided to use the Gazebo Harmonic simulator [60]. The distinctive feature of the ArduCopter flight controller software is its SITL (*Software in the Loop*) function [61]. The same system can be set up on a drone and in the simulator. All the commands and received information remain the same, allowing for faster, simpler, and more redundant testing.

First, a digital model of the drone had to be created. Gazebo requires SDF (*Simulation Description Format*) [62] files for its 3D rendering. They can be obtained from the CAD design, using the ACDC4Robot Fusion 360 extension [63]. It generates a URDF (*Unified Robot Description Format*) file [64], which can be converted into the final SDF file, using the built-in Gazebo tools. The drone model in the simulator is shown in Figure 4.1a. Its collision boxes are simplified to save the computing power (see Figure 4.1b).



(a) Drone in the simulator.

(b) Collision boxes.

Figure 4.1: Gazebo simulation environment.

The SDF file has to be appended with the following plugins to enable the ArduCopter SITL features:

- ArduPilotPlugin — main ArduCopter SITL plugin,
- gz-sim-lift-drag-system — lift and drag simulation plugin,
- gz-sim-joint-state-publisher-system — joint state publisher for the Gazebo simulator,
- gz-sim-apply-joint-force-system — rotors spin plugin.

Additionally, the Gazebo IMU and GPU LiDAR sensors [65] are added. Their parameters are adjusted to resemble their physical counterparts. The downside of the simulated LiDAR is its simplified implementation — it gathers all scans in the very same timestamp, so it is not possible to simulate a ranging smear, occurring in the real world.

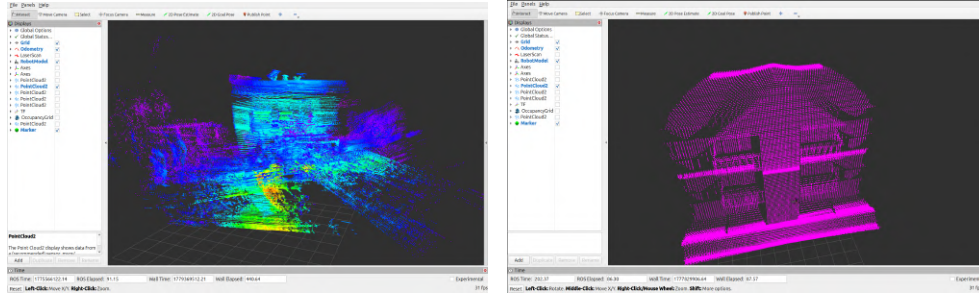
Sensor noise is calculated by the ArduCopter SITL layer. The chosen simulation parameters are presented in Table 4.1.

Parameter	Value
Scaling factor for simulated vibration from motors detected by the gyro	0.03
Gyro bias	0.003
Scaling factor for simulated vibration from motors detected by the accelerometer	0.2
Accelerometer bias	0.05
Simulated frame vibration frequency	65 Hz
Maximal frequency to use as baseline for adding motor noise for the gyros and accelerometers	65 Hz
Amplitude scaling of motor noise relative to gyro/accelerometer noise	0.15
Barometer noise in height	0.3 m
Scaling factor for simulated vibration from motors detected by the magnetometer	3
Number of satellites the GNSS has in view	11
GNSS accuracy	1.5 m
GNSS altitude drift error	0.05 m
Amplitude of the GNSS altitude error	2 m
Emulated wind speed	3 m/s
Emulated random wind variations	0.3 m/s

Table 4.1: Simulation parameters.

The simulation world is populated with models obtained from the Gazebo Fuel assets [66]. The OptiTrack model is taken from the UASwork open-source software repository [67]. The 3D models can be scanned by the emulated LiDAR, but it was too time and resource-consuming to simulate the intensity information of the reflected rays. Therefore, the intensity channel is not calculated and is set to a constant value (see Figure 4.2).

A separate ROS 2 package (`lidar_mapper_visualizer`) was created for data visualization. It works both with the simulation and (online) with the physical drone. Its main function is to display the gathered data, using the RViz2 tool [68]. It utilizes the URDF file to display the drone's model and calculate the required transformations (see Figure 4.3).



(a) Real-life data.

(b) Simulation data.

Figure 4.2: lidar_mapper_visualizer view.

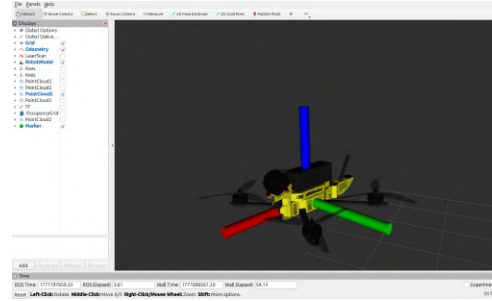


Figure 4.3: Drone's visualization in lidar_mapper_visualizer.

4.3 Data Acquisition

4.3.1 LiDAR Scans

Each sensor information is designed to be handled by a separate ROS 2 node, so it can be individually published and accessed for easier debugging. The LiDAR data is gathered by the `ldlidar_node` based on the `ldrobot-lidar-ros2` package [69]. It communicates with the LiDAR module over UART, applies initial filtering, and assembles the ROS 2 `LaserScan` messages [70]. These include the following information and are essential for the later deskewing:

- timestamp of the acquisition time of the first ray in the scan,
- start angle of the scan [rad],
- end angle of the scan [rad],
- angular distance between measurements [rad],
- time between measurements [s],
- minimum range value [m],
- maximum range value [m],

- range data [m],
- intensity data (integer value between 0 and 255).

The returned LiDAR scan is described in its own frame of reference, called *ldlidar_link*. However, the drone's position and orientation (the flight controller's frame — *base_link*) is in the world's frame — *map* (see Figure 4.4). The transformation between the *map* and *base_link* frames is represented by Equations 4.1–4.3.

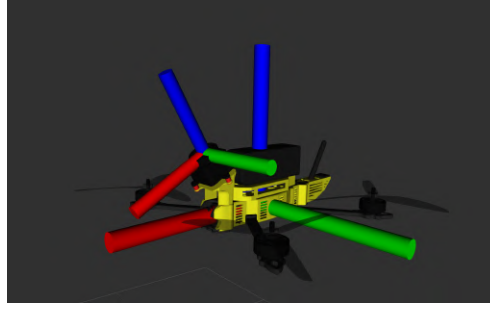


Figure 4.4: *ldlidar_link* and *base_link* frames of reference.

$$\begin{aligned}
 T_{base_link}^{map} &= \begin{bmatrix} R_{base_link}^{map} & p_{base_link}^{map} \\ 0_{1 \times 3} & 1 \end{bmatrix} = \\
 &= \begin{bmatrix} \cos \psi \cos \theta & \cos \psi \sin \theta \sin \phi - \sin \psi \cos \phi & \cos \psi \sin \theta \cos \phi + \sin \psi \sin \phi & x \\ \sin \psi \cos \theta & \sin \psi \sin \theta \sin \phi + \cos \psi \cos \phi & \sin \psi \sin \theta \cos \phi - \cos \psi \sin \phi & y \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.1)
 \end{aligned}$$

$$\begin{aligned}
 T_{ldlidar_link}^{base_link} &= \begin{bmatrix} R_{ldlidar_link}^{base_link} & p_{ldlidar_link}^{base_link} \\ 0_{1 \times 3} & 1 \end{bmatrix} = \\
 &= \begin{bmatrix} \cos \theta_{lidar} & 0 & \sin \theta_{lidar} & x_{lidar} \\ 0 & 1 & 0 & y_{lidar} \\ -\sin \theta_{lidar} & 0 & \cos \theta_{lidar} & z_{lidar} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.2)
 \end{aligned}$$

$$\begin{aligned}
 T_{ldlidar_node}^{map} &= \begin{bmatrix} R_{ldlidar_link}^{map} & p_{ldlidar_link}^{map} \\ 0_{1 \times 3} & 1 \end{bmatrix} = \\
 &= T_{base_link}^{map} T_{ldlidar_link}^{base_link} \quad (4.3)
 \end{aligned}$$

where:

- T — transformation matrix
- R — rotation matrix
- p — translation vector
- x, y, z — drone's position (in X, Y, and Z axes) in the *map* frame
- ϕ, θ, ψ — drone's orientation (roll, pitch, and yaw) in the *map* frame
- $x_{lidar}, y_{lidar}, z_{lidar}$ — LiDAR's position in the *base_link* frame
- θ_{lidar} — LiDAR's pitch orientation in the *base_link* frame

The drone's position is obtained from the GNSS antenna, the optical flow sensor, or the OptiTrack system, and is fused with other flight controller sensors. The LiDAR's pose in the *base_link* frame is taken from the CAD 3D model. The `lidar_mapper` node handles the transformations, using the `tf2` library [71]. Every time the main node receives a new pose data from the MAVROS node, a new transformation between the frames is published.

4.3.2 Drone Pose

The 3D pose of the drone is estimated by the ArduCopter's extended Kalman filter [72]. This algorithm fuses data from the IMU, GNSS, magnetometer, barometer, and other sensors to calculate a more reliable information than these sensors could provide on their own. The extended Kalman filter contains:

- state vector:
 - orientation quaternions,
 - velocity (north, east, down),
 - position (north, east, down),
 - delta angle bias (X, Y, Z),
 - delta velocity bias (Z),
 - wind vector (north, east),
 - Earth magnetic field vector (north, east, down),
 - body magnetic field vector (X, Y, Z),
- observations:
 - velocity (north, east, down),
 - position (north, east, down),

- true airspeed,
- magnetic flux (X, Y, Z),
- line of sight angular rate measurements from a downwards looking optical flow sensor (X, Y),
- range to terrain measurements,
- time varying parameters:
 - delta angle measurements in body axes (X, Y, Z),
 - delta velocity measurements in body axes (X, Y, Z).

The algorithm steps are described as follows.

1. Integration of IMU angular rates to calculate the angular position.
2. Conversion of IMU accelerations, using the angular position from body X, Y, Z to Earth north, east, down axes and correction for gravity.
3. Integration of accelerations to calculate the velocity.
4. Integration of velocities to calculate the position.
5. Estimated gyro and accelerometer noise (*State Prediction*) is used to estimate the error accumulation in the angles, velocities, and position calculated, using IMU data (*State Covariance Matrix*).
6. Steps 1 to 5 are repeated every time there is a new IMU data until a new measurement from another sensor is available.
7. When a GNSS (or other positioning system) measurement is available, the filter calculates the difference between the predicted position from 4 and the position from the GNSS (*Innovation*).
8. The *Innovation* from 7, the *State Covariance Matrix* from 5, and the GNSS measurement error are combined to calculate a correction to each of the filter states (*State Correction*).
9. The filter calculates the reduction in uncertainty due to the *State Correction*, updates the *State Covariance Matrix* and returns to step 1.

The estimated variables are then sent to the companion computer with the MAVLink protocol, and published in the `/mavros/local_position/odom` topic [73]. The main node subscribes to this topic to obtain the position and orientation of the LiDAR in space.

The communication between the flight controller and the companion computer is handled by the MAVROS node [74]. The transmission speed is set to the maximal 50 Hz update rate. To save the bandwidth, only the necessary plugins, like those

responsible for publishing the drone's pose, status, or navigation, are activated. This update rate leaves plenty of overhead for the scanning assembly in the real time regimes.

4.3.3 Scan Stacking

Each of the individual LiDAR scans is only a 2D slice of the scanned area. In order to get a full 3D scan, a push-broom method [75] is required. It works by moving a sensor with a limited field of view through the explored space to accumulate the desired information that would not be available otherwise (see Figure 4.5). Knowing the LiDAR's position and orientation, it is possible to transform the obtained scans to the world's frame of reference, effectively stitching them together over time.

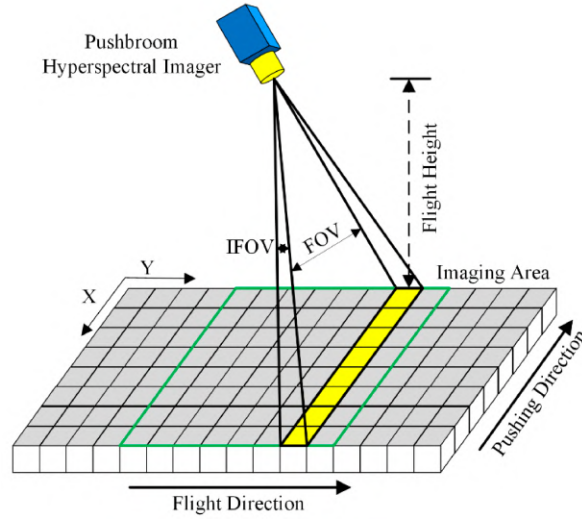


Figure 4.5: Push-broom scanning method [76].

However, the moving LiDAR generates scan distortions, caused by the rotating sensor's head (see Figure 4.6). To counteract this effect, the obtained readings have to be adjusted to the drone's pose at the time of taking the measurement. As the pose information, updated with the 50 Hz rate through MAVROS (and with the 10 Hz position update from the GNSS), does not match the exact timestamps of the individual scans, it has to be interpolated.

The `laser_geometry` library [78] is used for the pose interpolation and the scan deskewing. Additionally, it changes the format of the scan messages from `LaserScan` [70] to `PointCloud` [79], which is specialized for the point cloud data. The drone's position is easy to interpolate, using the linear interpolation (see Equation 4.4) between the last known position before a specific timestamp ($x/y/z_{prev}$) and the next one after it ($x/y/z_{next}$). The timestamp of each scanned point (t_{scan}) is deducted from the acquisition time of the first ray in the scan, time increment between the measurements, and the number of angle increments in a single sweep.

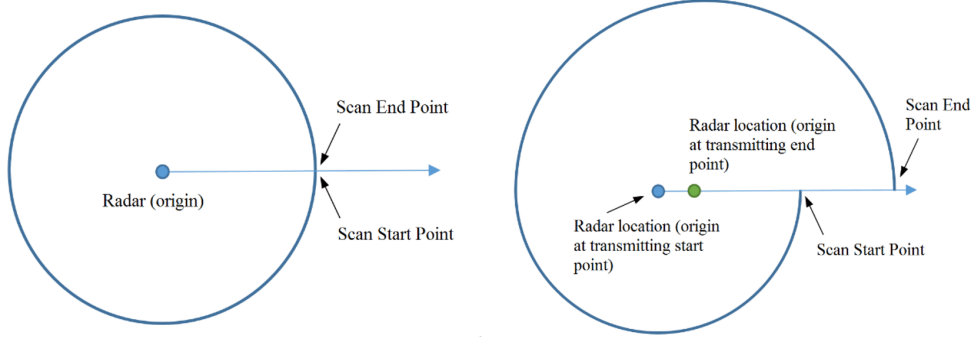


Figure 4.6: Moving LiDAR distortion principle [77].

The drone's poses are accumulated in a dedicated `tf2` buffer and can be used when the both positions are available for the interpolation.

$$\begin{cases} t = \frac{t_{scan} - t_{prev}}{t_{next} - t_{prev}} \\ x = x_{prev} + t(x_{next} - x_{prev}) \\ y = y_{prev} + t(y_{next} - y_{prev}) \\ z = z_{prev} + t(z_{next} - z_{prev}) \end{cases} \quad (4.4)$$

The orientation interpolation is more complex. It utilizes the SLERP (*Spherical Linear intERPolation*) algorithm [80] to maintain a constant angular velocity along the shortest path on the unit hypersphere between 2 quaternions. Equation 4.5 describes the interpolation between 2 orientation quaternions (q_{prev} and q_{next}) at a given scan timestamp.

$$\begin{aligned} SLERP(q_{prev}, q_{next}, t_{scan}) &= q_{prev}(q_{prev}^{-1}q_{next})^{t_{scan}} = \\ &= q_{next}(q_{next}^{-1}q_{prev})^{1-t_{scan}} = \\ &= (q_{prev}q_{next}^{-1})^{1-t_{scan}}q_{next} = \\ &= (q_{next}q_{prev}^{-1})^{t_{scan}}q_{prev} \end{aligned} \quad (4.5)$$

where:

$$e^{\mathbf{v}\Omega} = \mathbf{q}; \mathbf{v}^2 = -1 \quad (4.6)$$

$$\mathbf{q}^k = \cos(k\Omega) + \mathbf{v} \sin(k\Omega) \quad (4.7)$$

On the implementation side, all the published transformations are stored in the `tf2` buffer. To make sure that the pose can be interpolated, a special type of the ROS 2 subscriber, called `MessageFilter` [81], is used. It triggers the callback only when a new scan message is available within the same time threshold as the required transformations in the `tf2` buffer. Then, inside a callback, the scan's distortion is removed, and the scan message is converted to a point cloud message. The time

threshold is set to 0.11 s to accommodate the 10 Hz update frequency of both the LiDAR and the GNSS module.

The resulting point clouds are shown in Figure 4.7. They can be cumulatively added to create a 3D map of the drone’s surrounding.

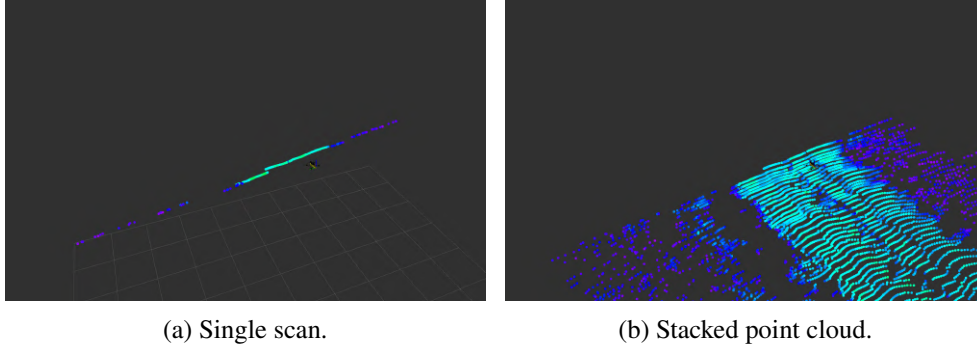


Figure 4.7: Resulting point clouds.

4.4 Filtration

4.4.1 Error Sources

The obtained scans suffer from multiple error sources. First, there are the LiDAR-induced ones, resulting from the imperfect range measurements. The LiDAR is a ToF (*Time of Flight*) sensor, which means it measures time between emitting a laser beam and receiving it back. As this time measurement is not ideal and the beam travels with a speed close to the speed of light, this inaccuracy introduces a small distance error. The photodiode detector itself is also prone to electrical and thermal noise, which skews its readings. 2D LiDARs usually consist of a single ToF sensor (1D LiDAR) mounted on a rotating shaft to gather multiple reading while it rotates. Usually the shaft is moved by a DC (*Direct Current*) motor, which does not have any angular position sensors, so it is assumed that its speed is constant. In reality, there is always a small jitter occurring, caused by the fluctuations in its power source and friction in the bearings. These effects affect the angular accuracy of the scanned points.

Moreover, the emitted laser beam tends to diverge with the distance travelled. Its shape is not a perfect cylinder, but rather a cone. This reduces the LiDAR’s angular resolution and makes it harder to distinguish between tightly-spaced objects or features. This phenomenon is also connected with the laser beam splitting on the edge of the scanned objects. In these situations, the sensor can receive 2 strong signals for the same angle and calculate an average distance between them, resulting in ghost detections. They can also occur because of the multipath propagation, caused by highly reflective objects (see Figure 4.8). When the returning beam does not take the shortest path back, it needs more time to get to the sensor, which is interpreted as an object being further away than it really is.

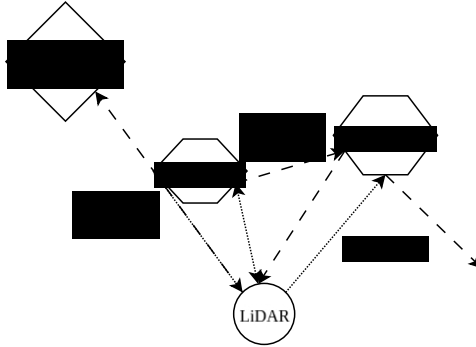


Figure 4.8: Detection disruption mechanisms.

LiDAR readings are heavily dependent on the reflectivity of scanned objects. More infrared-light-absorbent materials return lower intensities and might be accidentally filtered out in the post-processing. Specular surfaces pointing away from the sensor can also create blind spots, as the laser beam never goes back to the detector. Transparent objects, like glass, clear plastic, or water, are also invisible or look highly distorted for the LiDAR sensors.

LiDARs can report false detections in the presence of a direct, strong ambient light (like sunlight), but also when dust or water particles are suspended in the air. The ambient light can contain components with the same frequencies as the LiDAR's infrared laser uses. It can be confused with the actual emitted signal and then be visible as sparse targets. Dust particles reflect the laser beam back to the detector and can obstruct the LiDAR's field of view. Similarly, water particles, despite being translucent, can distort and skew the laser beam.

The second source of the errors affecting the proposed system are positioning uncertainties. The drone uses a small SoloGood M10-180C GNSS antenna, which promises only 1.5 m CEP (*Circular Error Probable*) accuracy. It can enhance its performance by utilizing the SBAS (Satellite-Based Augmentation System), which can typically increase the accuracy with errors below 1 m [82]. This is acceptable for regular navigational task, but for mapping it can introduce severe inaccuracies. When the drone's position is offset, the scans are as well.

The GNSS accuracy is influenced by the environment where the drone operates. When the drone flies close to buildings or tall trees, the GDOP (*Geometric Dilution Of Precision*) of the measured position increases, due to satellites arrangement on the horizon. Another problem of the GNSS modules is that they can be easily jammed or spoofed. As of this writing, it has become a growing issue — especially in the Baltic Sea region [83].

In the GNSS-denied scenarios, drones can obtain the positioning information from optical flow sensors. They can detect pixel patterns, using a camera and, based on that, calculate the change of their position (velocity) between the registered frames.

By integrating this data, the drone can track its movement from its starting point. In this project, the drone is equipped with the MicoAir MTF-02P optical flow sensor which, theoretically, can have a better accuracy than the GNSS module. However, optical flow sensors are dependent on the lighting conditions and the texture of the ground. If they are unable to find enough features on the captured image (for example, when the drone flies over a white, uniform floor), their accuracy drastically worsens. Similarly, when the objects on the ground move (like tall grass moved by wind gusts), it can be wrongly interpreted as the drone changing its position.

The altitude information comes either from the barometer or the rangefinder (the GNSS-based altitude can also be used, but it is less accurate, less reliable, and has a low update rate). The barometer returns a barometric altitude, which is relative to the drone's starting point. Its readings can be influenced by the local air temperature or wind, which cause them to fluctuate. The flight controller's SPL06 provides the accuracy of ± 0.5 m.

The rangefinder can achieve a much better accuracy when, the ground has a good enough reflectance (even up to 2 cm for the MicoAir MTF-02P). However, its main downsides are the limited maximal range (up to 6 m for the MicoAir MTF-02P) and the fact that its measurements are relative to the drone and not to its starting point. This means that on the same barometric altitude, the rangefinder can report different heights (see Figure 4.9). Therefore, the rangefinder cannot be used for the general mapping tasks, where all the scans have to be in the same global frame of reference. However, when the ground is considered to be flat across all the scanned area (for example, indoors, at car parks, or at squares), the rangefinder can prove to be more accurate than the barometer (at limited altitudes).

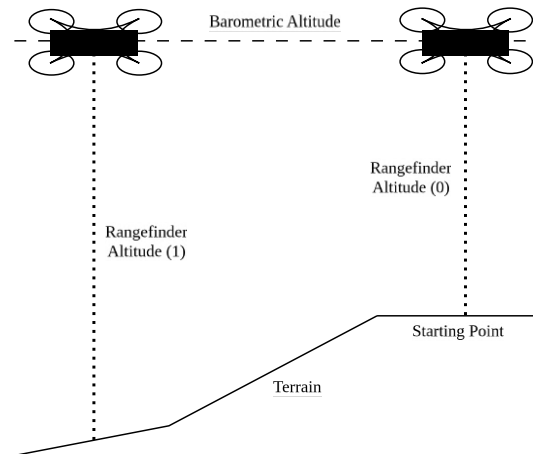


Figure 4.9: Barometric and rangefinder altitude.

4.4.2 Filtration Methods

The obtained scans are subject to a multistage filtration process to remove outliers and artefacts. The initial filter is applied just after a scan data is grabbed from the LiDAR and before the deskewing process (see Section 4.3.3). It only affects the points detected closer than 5 m away. First, it groups them into clusters based on their distance between each other. If the cluster contains more than 15 points, they are marked as valid. The clusters with less than 3 points are automatically discarded, unless their intensities are higher than a certain threshold. If the cluster has between 3 and 15 points, they are filtered based on their average intensity. This simple filter helps to discard the false-positive detections in the LiDAR's optimal ranging distance.

After first trials, it was noticed that, despite the initial filtering, the LiDAR still registers some false-positive detections. They are especially visible, when the strong direct sunlight is present (see Figure 4.10). These detections can be described as low-intensity, sparse points appearing above the drone, even in open areas.

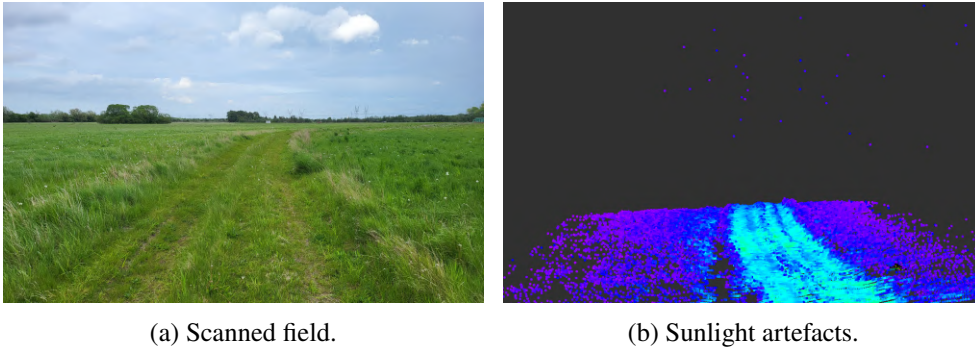


Figure 4.10: Encountered sunlight artefacts.

These outliers are removed with a second-stage SOR (*Statistical Outlier Removal*) filter [84]. For each point in the point cloud, a mean distance from its k -nearest neighbours is calculated. The point is being rejected if its mean distance exceeds a threshold, based on the standard deviation of the global inter-neighbours distances (see Figure 4.11).

Figure 4.12 presents the effects of this filtration — the sparse outliers are effectively filtered out. Typically, this filter is used on already obtained point clouds. This would be acceptable for the mapping tasks, where the drone would first gather the scans and process them afterwards. However, for the online navigation, the rapidly expanding point clouds would increase the computational cost of this operation over time. That could introduce a risk of getting outside the real-time operation regime. That is why it was decided to apply the SOR filter only to the individual 2D scans, before stitching them to the rest of the growing point cloud. This solution introduces a minimal computational overhead, and still delivers satisfactory results.

Sparse outliers could also be filtered out with a radius outlier removal filter [85]. It keeps the points which have a specified number of neighbours within a specified

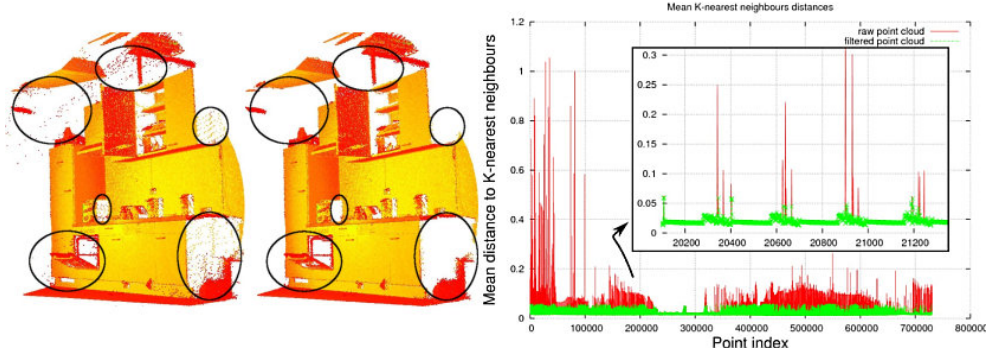


Figure 4.11: SOR filtering example [84].

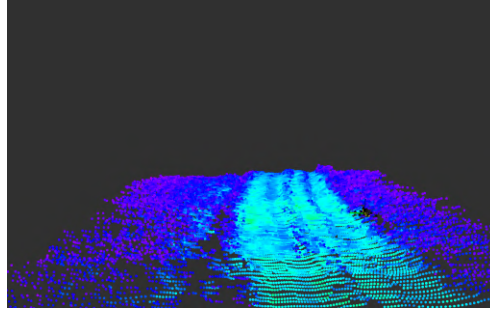


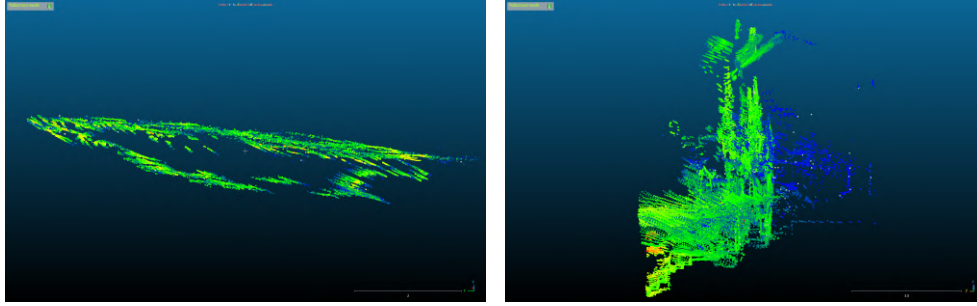
Figure 4.12: SOR filter effects ($k = 50$, threshold = 1σ).

radius around them. Its main advantage is a low computational cost, but it was noticed that it is too strict for the gathered, low-density point clouds. The hardcoded filtering radius makes it also harder to adapt in different environments.

The low density of the obtained point clouds is also the reason why a voxel grid filter [86] was not used for the filtration. It downsamples the collected data by dividing the space into voxels (cube-shaped 3D pixels) and approximating the points contained in them with their centroid. It outputs averaged, but significantly sparser point clouds, which is not acceptable when they already contain a limited amount of information from the beginning.

The positioning errors result with doubled, ghost surfaces, occurring when the same area is scanned more than once (see Figure 4.13). When the drone's pose is estimated incorrectly, the associated scans are offset from the ground truth as well.

These problems can be mitigated by enhancing the pose estimation. In terms of the GNSS positioning, there is not much that can be done, as it is limited by the sensor's characteristics. The u-blox M10 chip is capable of utilizing SBAS when it is available, but an RTK system could further increase the accuracy, from the submetre to the centimetre level. Taking this under consideration, the proposed system should rather be used away from tall or tightly spaced buildings. The operator should mind the number and the quality of available GNSS satellites — ideally, the GDOP should be below 2. In certain situations, using the optical flow sensor can give better results.



(a) Double ground.

(b) Double wall.

Figure 4.13: Ghost surfaces.

The pose of the drone is determined from multiple sensors, using the extended Kalman filter (see Section 4.3.2). It can be further improved by filtering the raw sensor readings before the Kalman filter stage. The ArduCopter applies low-pass and notch filters on the IMU data. Combined, they greatly reduce the influence of the motor vibrations on the sensor. Even though the flight controller is suspended on the silicon dampers, the unwanted vibrations can still reach it. The mentioned notch filter uses the angular velocity data from the ESCs over the bidirectional DShot to isolate these vibration frequencies and apply the filter exactly where it is needed (see Figure 4.14). This solution introduces a small phase lag to the control loop, but it is within the acceptable levels.

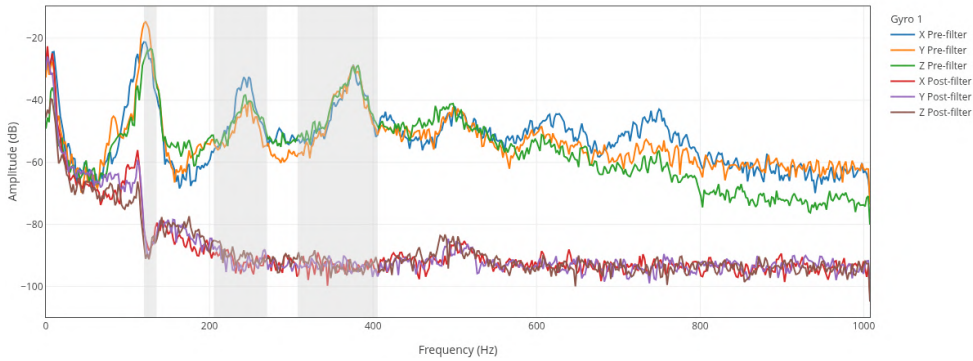


Figure 4.14: ESC-telemetry-based IMU notch filtering.

Unfortunately, despite applying the described sensor filtration methods, the ghost surfaces kept appearing. An additional scan post-processing was required.

This issue is addressed with the help of the OctoMap [87]. It is a framework for a volumetric 3D environment models generation, based on octrees [88]. It uses a probabilistic occupancy estimation to efficiently describe the mapped surroundings.

The OctoMap divides a space into occupied, free, and unknown voxels. Usually the same type of voxels are close to each other, so they can be gathered into clusters to save memory. The octree describes a recursive subdivision of the voxels into

8 subvolumes, performed until the resulting subvoxel is uniform or reaches its minimal size (see Figure 4.15). This method is incredibly memory-efficient, which makes it perfect for real-time applications on edge devices. Additionally, the resolution of the resulting maps can be easily scaled for given needs.

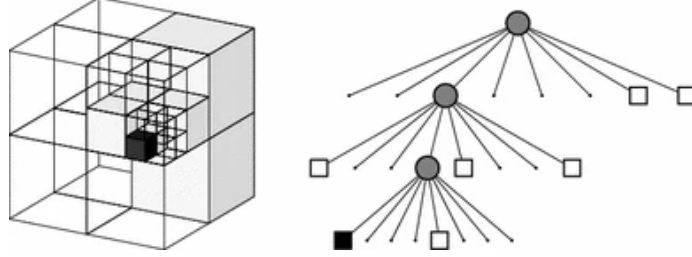


Figure 4.15: Octree storing OctoMap space [87].

The OctoMap also acts as a noise filter. Every final subvoxel (a leaf of the octree) is given a probability of being occupied. This probability is updated with every new scan and holds a memory of previous ones (see Equation 4.8). To be more computationally efficient, the OctoMap framework uses the log-odds notation, as additions are faster than multiplications (see Equations 4.9 and 4.10). Finally, a threshold is applied to the calculated probability (see Equation 4.11) to further decrease the size of the resulting map while preserving its quick adaptability.

$$P(n|z_{1:t}) = \left(1 + \frac{1 - P(n|z_t)}{P(n|z_t)} \frac{1 - P(n|z_{1:t-1})}{P(n|z_{1:t-1})} \frac{1 - P(n)}{P(n)} \right)^{-1} \quad (4.8)$$

$$L(n) = \log \left(\frac{P(n)}{1 - P(n)} \right) \quad (4.9)$$

$$L(n|z_{1:t}) = L(n|z_{1:t-1}) + L(n|z_t) \quad (4.10)$$

$$L(n|z_{1:t}) = \max(\min(L(n|z_{1:t-1}) + L(n|z_t), l_{max}), l_{min}) \quad (4.11)$$

where:

- $P(n|z_{1:t})$ — probability of a leaf node n to be occupied, given the sensor measurements $z_{1:t}$
- z_t — current measurement
- $P(n)$ — prior probability (usually 0.5)
- $P(n|z_{1:t-1})$ — previous estimate
- $P(n|z_t)$ — probability of voxel n to be occupied, given the measurement z_t
- $L(n)$ — log-odds notation of $P(n)$

- l_{min} — lower threshold for $L(n|z_{1:t})$
- l_{max} — upper threshold for $L(n|z_{1:t})$

Thanks to this probability filter, the OctoMap can adjust the previously obtained voxels based on the new incoming data. When one of the scan sweeps is registered in a different place than the following ones, its probability of being placed correctly drops over time — the new scans mark the previously occupied voxel as free (see Figure 4.16). This property of the OctoMap means that it can filter out dynamic objects from the point cloud, like walking people or moving vehicles. However, as it requires multiple scans to do that, a ghosting effect may occur.

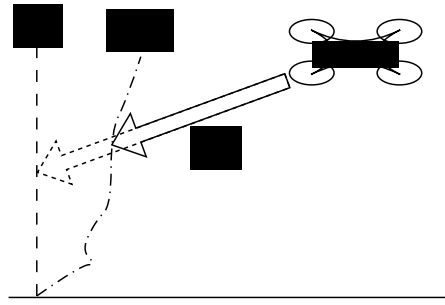


Figure 4.16: OctoMap correction.

The presented project uses the `octomap_server` [89] ROS 2 implementation of the OctoMap. The accumulated point cloud after the SOR filtration is fed to the input of the OctoMap. Given the positioning accuracy and the computational capabilities of the companion computer, the minimal voxel size is set to 15 cm. Some of the obtained OctoMaps are presented in Figure 4.17. The colour channel there represents the voxel occupancy probability.

Every time a new update for the OctoMap is available, a callback is triggered. There, the OctoMap is transformed into the `PointCloud` ROS 2 format and cropped to a $20\text{ m} \times 20\text{ m} \times 20\text{ m}$ window to save the processing time. The intensity channel is replaced with the occupancy probability information. The final effect of the filtration is presented in Figure 4.18.

4.5 Mapping

The described solution is capable of creating small, low-resolution maps. Because of the companion computer’s limited capabilities, it is optimized for real-time efficiency to work online on the drone. The mapping task, however, can be done as a post-processing step. The drone scans a desired area first, then lands and sends the gathered data for analysis on a more powerful device.

When the map is assembled offline, the resolution of the OctoMap can be increased to 5 cm. The map windowing can be completely abandoned to keep the full size of

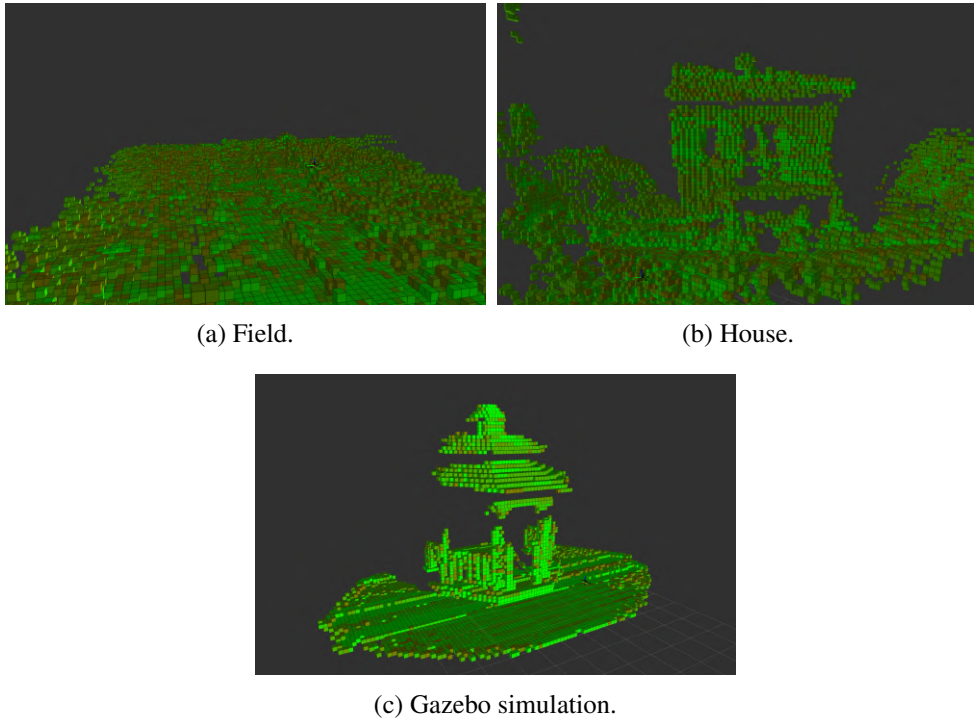


Figure 4.17: Obtained OctoMaps.

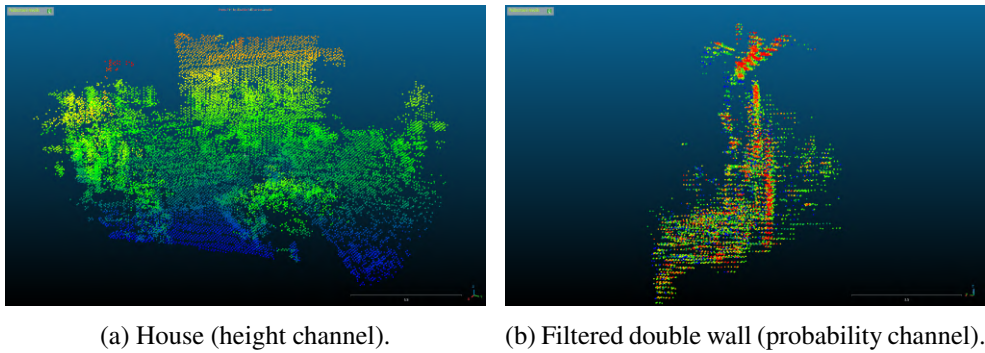


Figure 4.18: OctoMap filtering results in different colour channels.

the scanned area. As can be seen in Figure 4.19, the increased resolution preserves more details, but it is still heavily dependent on the positioning quality (the scans here were obtained with the OptiTrack system).

The OctoMap filtering results in less dense point clouds than the raw scans can provide. Furthermore, it requires multiple scan sweeps to properly filter out inconsistencies, which are still not guaranteed to be removed completely. This method works better with simple flat surfaces than with detail-rich objects.

In the field of LiDAR mapping, the point clouds often have to be manually refined. A human editor can remove outliers and adjust the position of particular

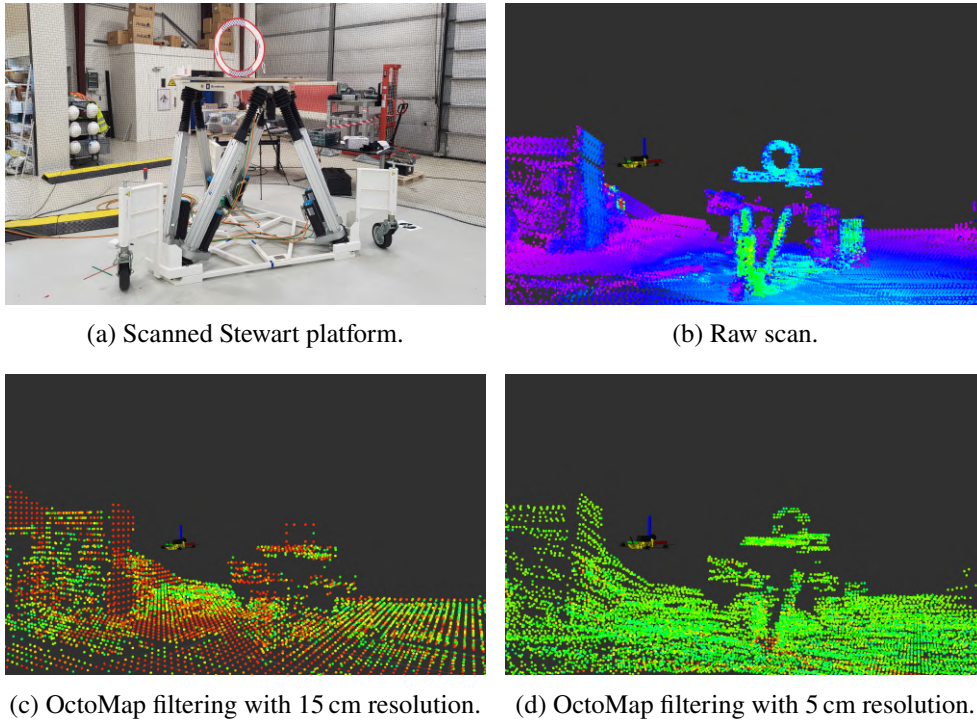


Figure 4.19: OctoMap resolution change.

scans. Sometimes, automated methods can be used as well. One of these methods is the ICP (*Iterative Closest Point*) algorithm [90]. It minimizes a difference between 2 point clouds. By iteratively applying transformations to the source scan, it minimizes the sum of squared Euclidean distance between its and the target scan's position (see Figure 4.20).

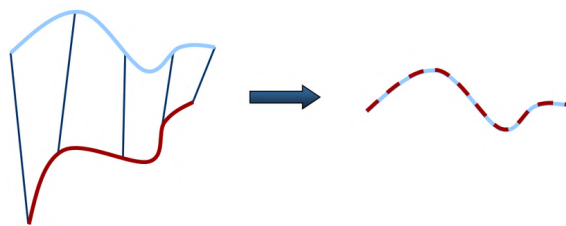


Figure 4.20: ICP method adjustment [91].

In the post-processing pipeline, the source point cloud for the ICP correction is obtained from a buffer of 25 successive scans. Within this buffer, 15 scans overlap with the previous one (see Figure 4.21). These buffers act as a fake 3D LiDAR data (although, with a much lower update frequency). Next, the ICP point-to-plane algorithm [92] tries to adjust the pose of the source point cloud to the target point cloud. The initial transformation guess is taken from the last known drone's pose of

the last scan in the buffer. Finally, the transformed source point cloud is added to the target point cloud, which is then used in the next iteration.

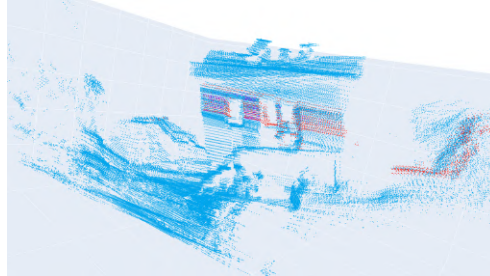


Figure 4.21: Buffered scans (red) for ICP correction.

The initial tests of this correction method were conducted on a house scan, characterized by double, ghost walls, caused by the significantly low positioning accuracy. The results of the ICP correction are shown in Figure 4.22. The raw scan visualization uses the intensity colour channel, while the corrected point cloud is displayed with the height colour channel. The scan distortion is not resolved perfectly, but the double walls are successfully merged into one.

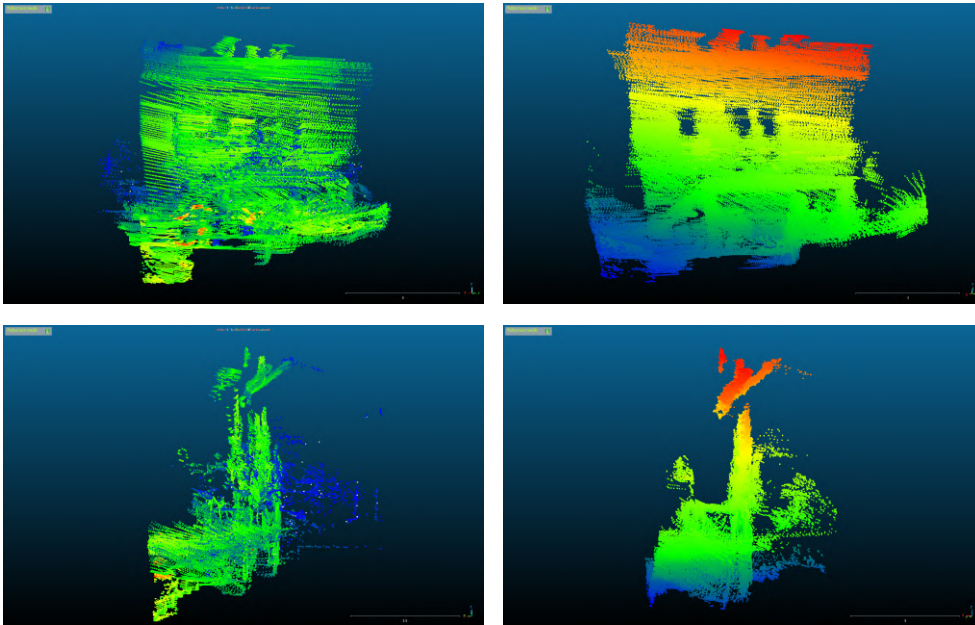


Figure 4.22: House raw scan (left) ICP correction (right).

4.6 Navigation

The previously described OctoMap filtration, while not ideal for mapping, is perfect for the navigation tasks. It is computationally lightweight, adaptive and well-suited for real-time operation regimes.

In this project, the EGO-Swarm [93] trajectory planner is used for the drone navigation and collision avoidance. Although it was primarily designed for drone swarms, it still works perfectly with a single UAV. The EGO-Swarm is a more robust evolution of the previous EGO-Planner (*ESDF-free Gradient-based lOcal Planner*) [94]. Typical trajectory planners usually use an ESDF (*Euclidean Signed Distance Field*) globally to evaluate gradient magnitude and direction, when refining the spline control points. However, this mathematical operation significantly increases the computation time. Figure 4.23 compares an area covered by the ESDF calculations with a space actually used in the trajectory optimization. Therefore, the authors proposed a gradient-based spline optimizer which generates a collision cost by comparing a trajectory inside obstacles with a guiding collision-free one. This approach makes this planner an ideal solution for aerial drones, as it is computationally faster than other ESDF-based ones by over an order of magnitude.

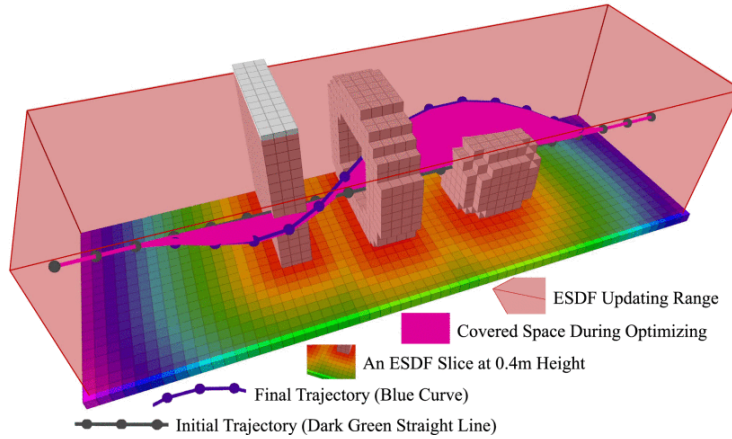


Figure 4.23: ESDF updating range [94].

The EGO-Planner finds the initial rough path, using the A* algorithm [95]. This efficient planner tends to generate trajectories close to obstacles (it finds the shortest path to the target). Therefore, it is further refined with the gradient-based optimizer. The following penalties are taken into the consideration during the optimization process:

- smoothness penalty — makes the trajectory smoother by minimizing the squared acceleration and jerk of the B-spline control points,
- collision penalty — moves the B-spline control points away from the encountered obstacles with a safety clearance,

- feasibility penalty — restricts the higher order derivatives of the B-spline on every dimension to ensure its generation within the given constraints.

The concept behind the gradient-based optimization is shown in Figure 4.24. The used numerical solver is also refined to eliminate computational bottlenecks.

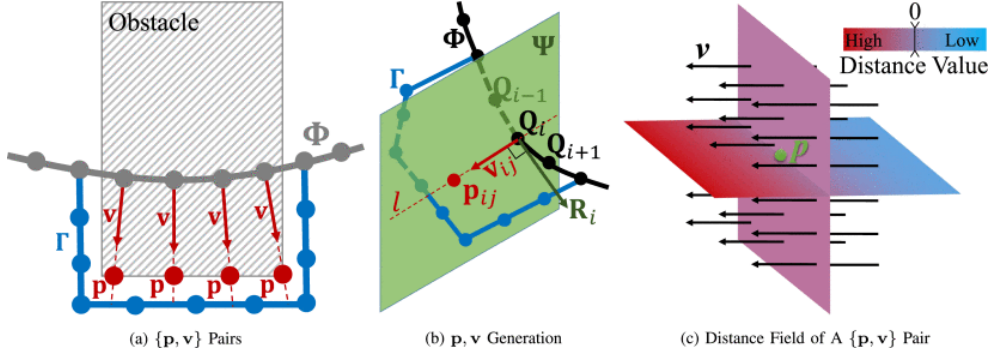


Figure 4.24: EGO-Planner optimization concept [94].

The planning process consists of several stages. First, the program initializes and waits for a target position to be specified. Next, a new trajectory is generated based on the currently available map. The drone tries to follow it within the speed and acceleration limits. If a new obstacle appears during the flight, the planner adjusts the trajectory to avoid it, without interrupting the drone’s movement. However, if an occupied region of the map suddenly appears closer than the specified distance threshold, the drone performs an emergency stop. This state is also triggered when the trajectory replanning exceeds a time limit. Then, the drone tries to stop as quickly as it is possible and recompute its path from there, with more relaxed parameters.

The original implementation of the EGO-Swarm was written for the previous version of ROS, therefore the author forked the `ego-planner-swarm` repository [96] and refined it to match the ROS 2 standards. As a part of the implemented changes, the input depth camera image was replaced with the point cloud and the package’s parameters were adjusted accordingly.

Each of the points from the OctoMap-filtered point cloud is inflated to a $0.5 \text{ m} \times 0.5 \text{ m} \times 0.5 \text{ m}$ cube (marked with a white colour in Figure 4.25). This operation provides a safety margin of a minimal acceptable distance between a drone and obstacles. The planning horizon for the optimizer is set to 10 m. Because of the LiDAR being mounted at the 30° angle, the drone has a blind spot when travelling up and forward. To address this issue, an initial yaw ‘look-around’ manoeuvre is added before every trajectory optimization start. This 360° spin is also performed when the drone enters the emergency stop state. The additional data about the current environment helps with a safe path replanning and moving away from the encountered obstacles. Furthermore, the inflated points are further expanded in the Z axis to 125% of their previous size. The maximal drone speed and acceleration are limited to 0.6 m/s and 0.2 m/s^2 , respectively.

To save the memory, the maximal size of the map used by the planner is set to $45\text{ m} \times 45\text{ m} \times 30\text{ m}$. It is locally updated only within the $8\text{ m} \times 8\text{ m} \times 7.5\text{ m}$ range around the drone. These constraints keep the algorithm efficient, so it can be used on the actual, real-life drone.

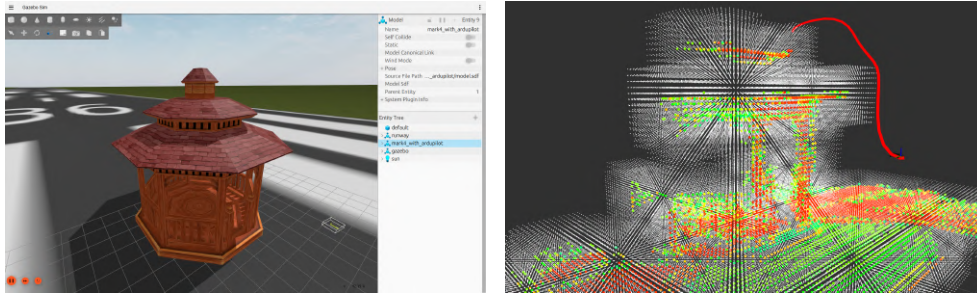


Figure 4.25: Generated trajectory (red line) around simulated gazebo.

5 Results

5.1 Mapping

5.1.1 LiDAR Accuracy Test

To test the LiDAR's accuracy, it was remounted to be perpendicular to the ground. Then, the drone was put on the floor, in front of a white wall (see Figure 5.1). The drone was gradually moved away from the wall (in 30 cm steps, up to approximately 3 m away) and the LiDAR's measurements for 0° heading were logged. Unfortunately, the full range of the sensor could not be evaluated, because of lack of the available testing space. The ground truth distance was measured with a measuring tape after placing the drone manually, so it is not precise. However, its error lays within the LiDAR's declared accuracy of 3 cm (2 cm on distances up to 2 m). Therefore, this experiment checks if there are any significant deviations from the LiDAR's specification.

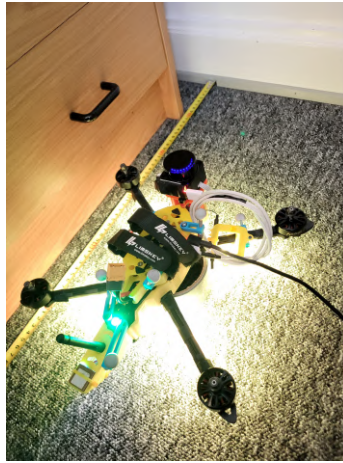


Figure 5.1: LiDAR accuracy test setup.

Figure 5.2 presents the test results. The first plot compares the ground truth data with the measured distance. The average, minimal, and maximal recorded distances are shown there. The second plot displays the average, minimal, and maximal errors between the ground truth and the measured data.

The performed test shows that the chosen LiDAR's accuracy is acceptable for general mapping tasks, like building or landmark scanning. Its overall average error is 0.7 cm on the distances up to 3 m (with the overall average standard deviation of 0.08 cm), but its spread increases over distance. These numbers can be interpreted to match the LiDAR's specification.

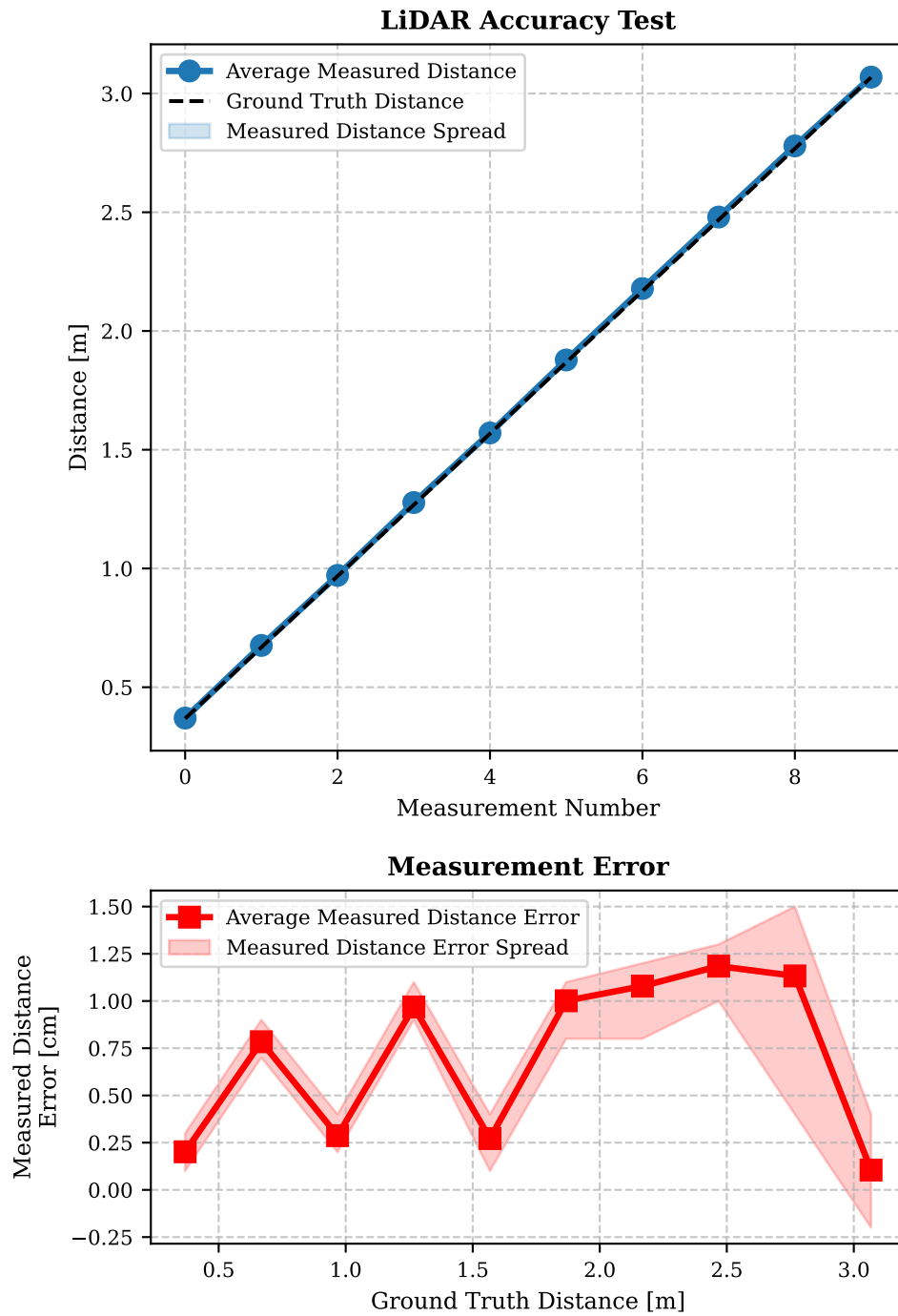


Figure 5.2: LiDAR accuracy test results.

5.1.2 Positioning Accuracy Test

The positioning test was performed for the following systems:

- GNSS module with barometer,
- optical flow sensor with rangefinder,
- OptiTrack motion capture system.

The drone was put into the loiter (position hold) mode and flew into the initial position. There, it was left without changing the radio inputs, and the drift around the set position was registered. The outdoor flights were performed in a wind-covered area, where the GNSS HDOP (*Horizontal Dilution Of Precision*) parameter was registered to be approximately 2. The ground was covered in grass and the sky was cloudy, so the lighting conditions were not ideal.

The results are shown in Figure 5.3. The most stable readings were obtained from the OptiTrack system. The GNSS module and the optical flow module drift around the setpoint in the XY plane a little, but within a sane margin. In terms of the altitude readings, large fluctuations in the barometer data can be seen. The rangefinder does also experience some, but with much lower magnitudes. The standard deviations of the obtained data are presented in Table 5.1.

Sensor	Standard Deviation [m]
GNSS XY Plane	0.047
Barometer Altitude	0.222
Optical Flow XY Plane	0.05
Rangefinder Altitude	0.052
OptiTrack XY Plane	0.003
OptiTrack Altitude	0.005

Table 5.1: Positioning data standard deviations.

The tests were repeated in an open area where the HDOP parameter was better (approximately 1.2). However, this time the drone experienced moderate winds, so their influence on the horizontal positioning could be checked as well. The result are presented in Figure 5.4 and in Table 5.2.

Sensor	Standard Deviation [m]
GNSS XY Plane	0.089
Barometer Altitude	0.054
Optical Flow XY Plane	0.091
Rangefinder Altitude	0.028

Table 5.2: Positioning data standard deviations in windy conditions.

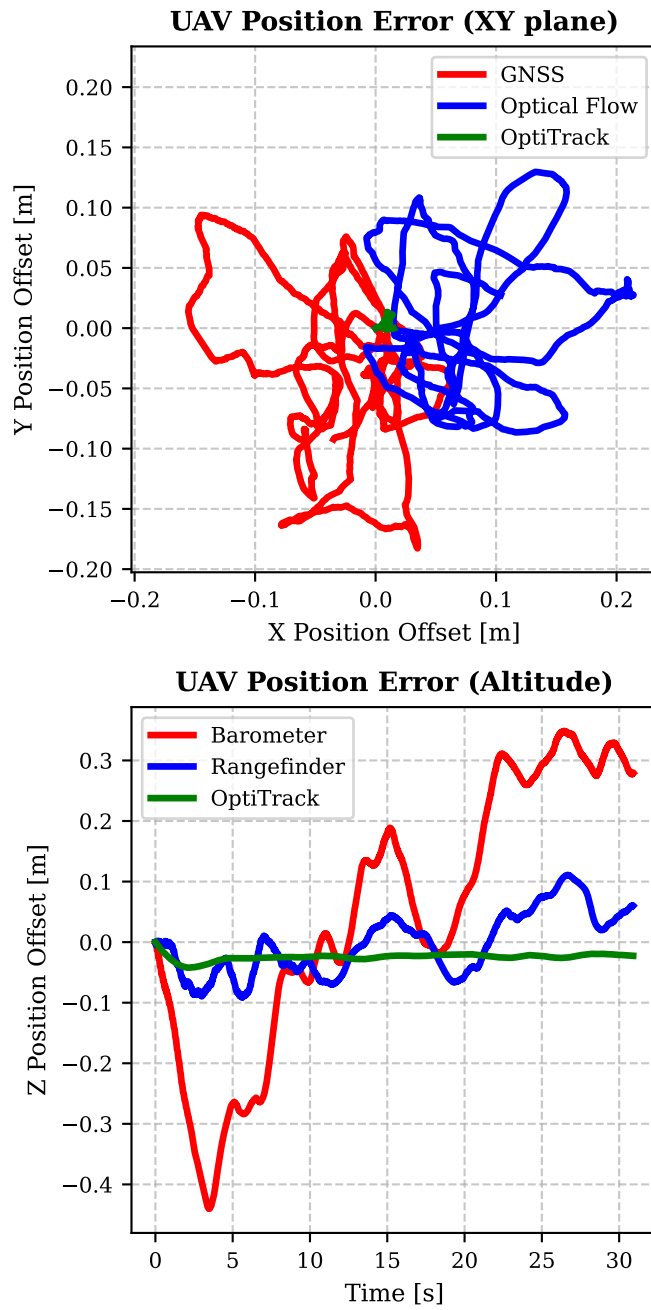


Figure 5.3: Positioning error test results.

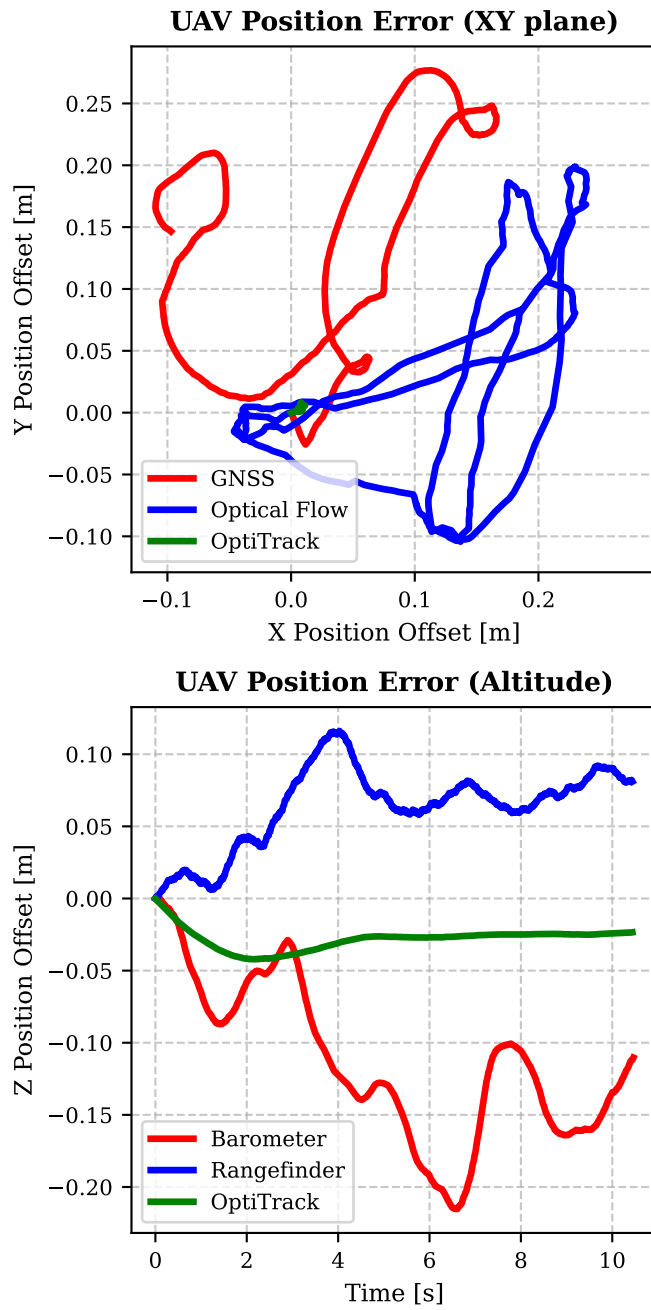


Figure 5.4: Positioning error test results in windy conditions.

The obtained data helps to determine the quality of the used positioning sensors. The OptiTrack promises the best positioning results, and it can be used to model an RTK system. The optical flow sensor seems to have similar capabilities as the GNSS module. However, the use of the rangefinder can increase the altitude accuracy in certain conditions, compared to the barometer sensor. The moderate wind was moving the drone around, but it did not cause any significant changes in the barometer readings, which proves the drone's resilience to outdoor conditions.

This experiment evaluates the drone's positioning accuracy under environmental and self-induced disturbances. An additional static test (for example, with the drone mounted on a pole a few metres above the ground) could also be conducted to measure raw sensor accuracy. However, due to available equipment limitations, it could not be performed within the given project time.

5.1.3 Point Clouds

The point clouds obtained after the ICP correction step were converted into meshes, using the Poisson reconstruction [97][98][99] in the CloudCompare processing software [100]. This process helped with evaluating the accuracy of the gathered point clouds. The results are presented in Figures 5.5–5.15 with the height colour channel. Each of them contains a photo of an object / location, a point cloud after the ICP correction, and an exported mesh.

The tests conducted with the OptiTrack simulate the usage of a high-precision positioning system, like the RTK. Their main purpose was to establish the baseline of the system's performance. The GNSS was compared with the optical flow sensor in the same environments. The following scenarios were evaluated:

- Optitrack positioning:
 - floor — a smooth surface benchmark,
 - single tool box — a small object with known dimensions, placed close to the floor,
 - stacked tool boxes — an easily-distinguishable, larger object with known dimensions,
 - Stewart platform — an object with a complex geometry,
 - clutter — multiple objects with various geometries,
- GNSS positioning:
 - house — a common mapping task, where the GNSS system is prone to inaccuracies,
 - tree — an object that often sways in the wind, causing the ghosting effect,
- GNSS / optical flow positioning:
 - field — a flat surface, where the 2 sensors can be compared,

- riverside bridge — a relatively flat surface, with a complex-geometry object present.

Floor (OptiTrack Positioning)

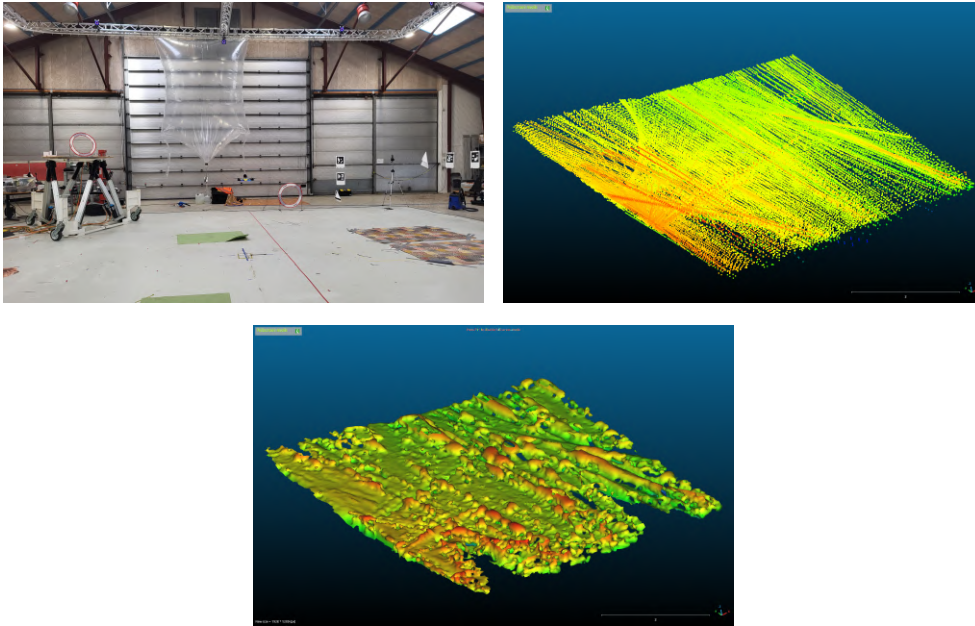


Figure 5.5: Floor scans (OptiTrack positioning).

First, a floor inside the OptiTrack system was mapped without any objects on it. The obtained scan is not ideal, but no double surface effect occurred in the final point cloud. The scans were gathered by flying the drone back and forward with occasional heading changes.

Single Tool Box (OptiTrack Positioning)

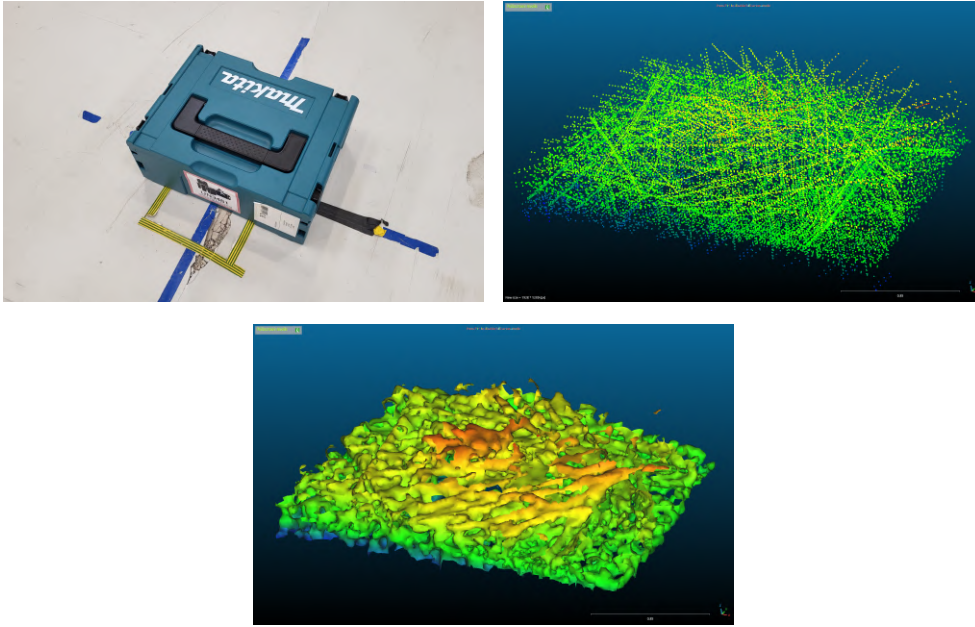


Figure 5.6: Tool box scans (OptiTrack positioning).

A single plastic tool box (dimensions: $39.2\text{ cm} \times 29.2\text{ cm} \times 16.2\text{ cm}$) was placed flat on the floor to check the sensitivity of the proposed system. This time, the drone circled around it, while simultaneously changing its altitude. This allowed to register multiple overlaying scans from various angles. The maximal movement speed of the drone was set to 1000 cm/s horizontal, 250 cm/s up, and 150 cm/s down.

Although, the tool box is noticeable in the final mesh, its shape was not extracted correctly, as it drowned in the surrounding noise.

Stacked Tool Boxes (OptiTrack Positioning)

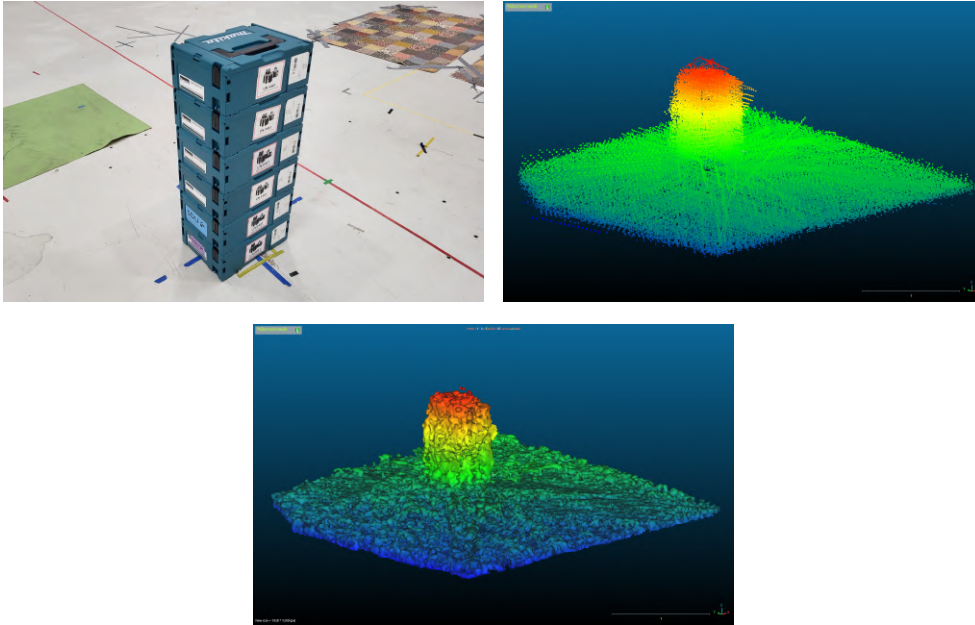


Figure 5.7: Stacked tool boxes scans (OptiTrack positioning).

In the next test, 6 tool boxes were stacked on top of each other (dimensions: $39.2\text{ cm} \times 29.2\text{ cm} \times 97.2\text{ cm}$) to check the accuracy of the system when a larger object is present. Once again, the drone was circulating around it with a varying altitude, but its maximal speed was significantly reduced to 100 cm/s horizontal, 25 cm/s up, and 15 cm/s down to obtain denser point clouds.

The lower scanning speed resulted in a denser and more precise point cloud. Some minor artefacts are still noticeable there, but they were successfully removed during the mesh reconstruction process. However, the size of the extracted mesh is roughly $49\text{ cm} \times 44\text{ cm} \times 96\text{ cm}$, so the horizontal dimensions are significantly bloated. This mismatch can be explained by the multiple errors accumulated during the circular scanning flight. Because of that, the shape of the stacked boxes is not perfectly preserved.

Stewart Platform (OptiTrack Positioning)

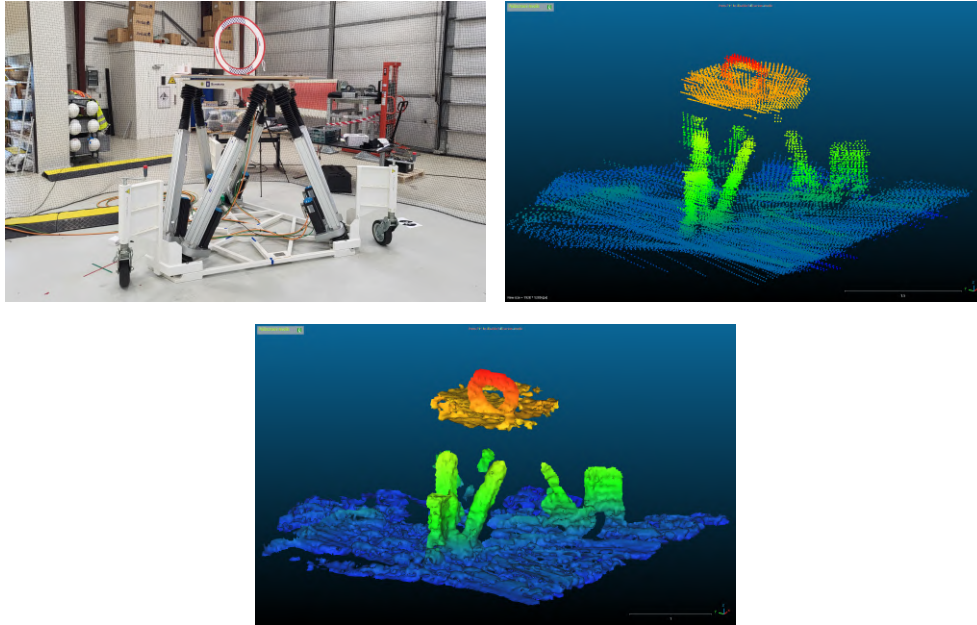


Figure 5.8: Stewart platform scans (OptiTrack positioning).

A Stewart platform was scanned as an example of an object with a complex geometry. The platform is 171.5 cm tall, and its linear actuator's main body has dimensions of approximately $15\text{ cm} \times 15\text{ cm} \times 99\text{ cm}$.

The overall geometry of the platform was captured successfully. The height of the scan was measured to be roughly 160 cm, while the part representing the linear actuator dimensions is approximately $16\text{ cm} \times 14\text{ cm} \times 95\text{ cm}$. These measurements roughly match with the real object, but some artefacts are still visible. The model is incomplete, because a part of it was in the LiDAR's blind spot. As the scan was performed by changing the drone's altitude while facing the platform, its top plate covered the upper part of the linear actuators. This blind spot could be, however, eliminated by performing a yaw spin near it. Nevertheless, the fabric ring, placed on the top, was captured successfully.

Clutter (OptiTrack Positioning)

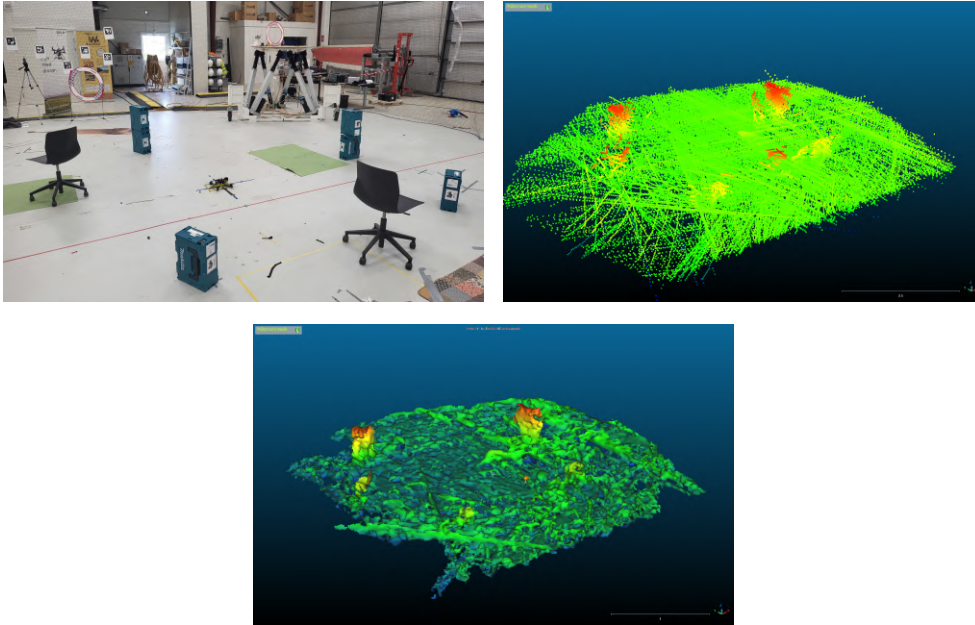


Figure 5.9: Clutter scans (OptiTrack positioning).

A manual flight was performed in a cluttered environment, filled with tool boxes and chairs positioned randomly around the drone. 6 separate objects were counted on the point cloud, but their details were not captured. Therefore, they all appear as blobs and are hard to identify.

House (GNSS Positioning)

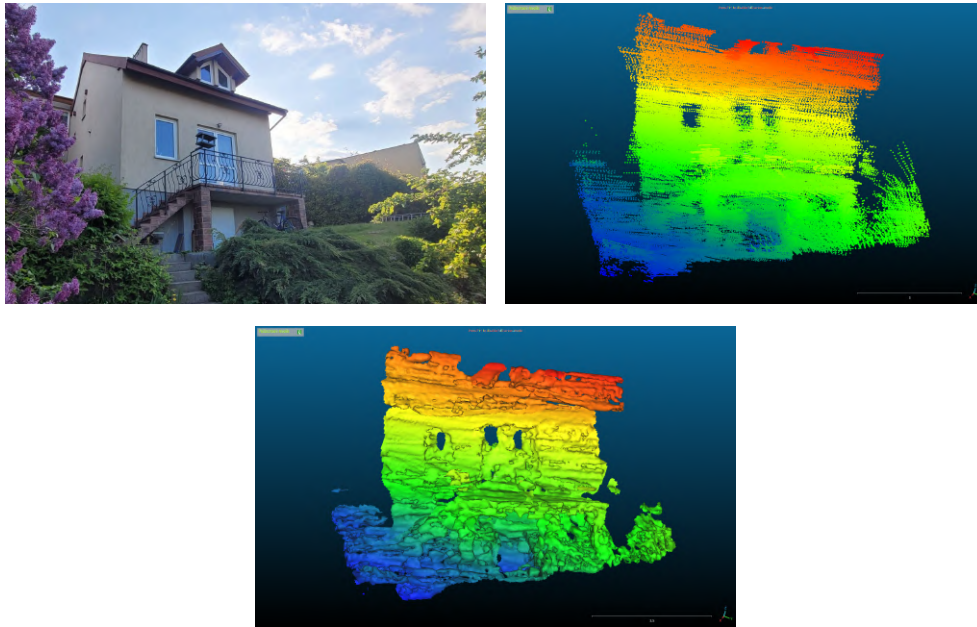


Figure 5.10: House scans (GNSS positioning).

The direct vicinity of a scanned house meant worse GNSS positioning accuracy. The positioning drift caused ghost wall effect in the point cloud. The ICP correction managed to fix this problem, but the overall scan is skewed and not precise. However, the balcony and the bird feeder are still recognizable on the resulting mesh. In this case, the scans were obtained by flying the drone up and down close to the building.

Tree (GNSS Positioning)

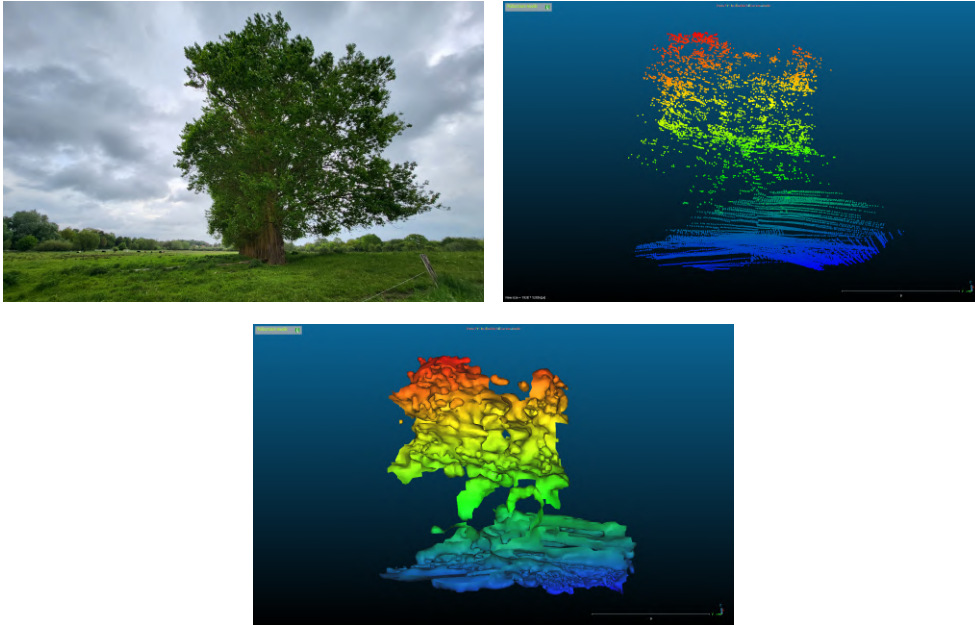


Figure 5.11: Tree scans (GNSS positioning).

Scanning a tree resulted in a sparse and not precise point cloud, because of the leaves moving on wind. Despite the missing details, the general shape of the tree was captured successfully. Similarly to the house case, the scans were gathered by changing the drone's altitude.

Field (GNSS / Optical Flow Positioning)

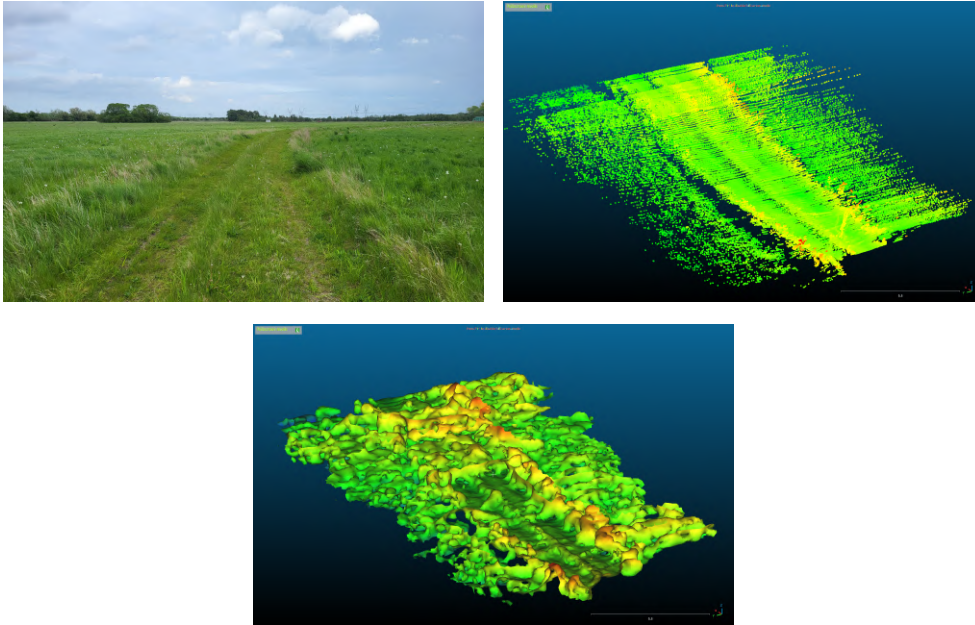


Figure 5.12: Field scans (GNSS positioning).

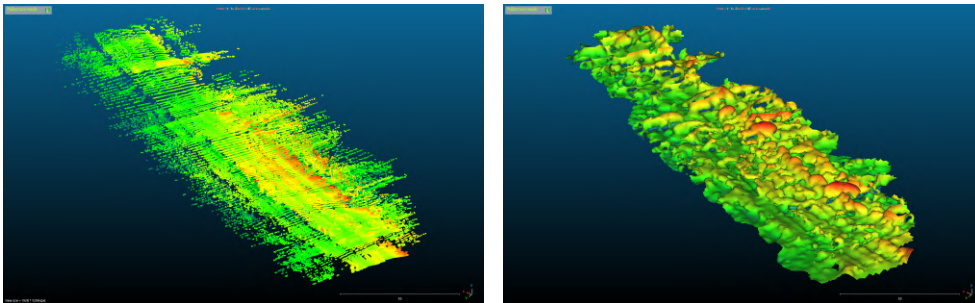


Figure 5.13: Field scans (optical flow positioning).

A relatively flat field with a distinctive dirt road was scanned to compare the GNSS and the optical flow positioning systems. In both of the extracted meshes the noise is noticeable, but the dirt road was not lost in it. Here, the drone was commanded to move back and forward over the field.

Riverside Bridge (GNSS / Optical Flow Positioning)

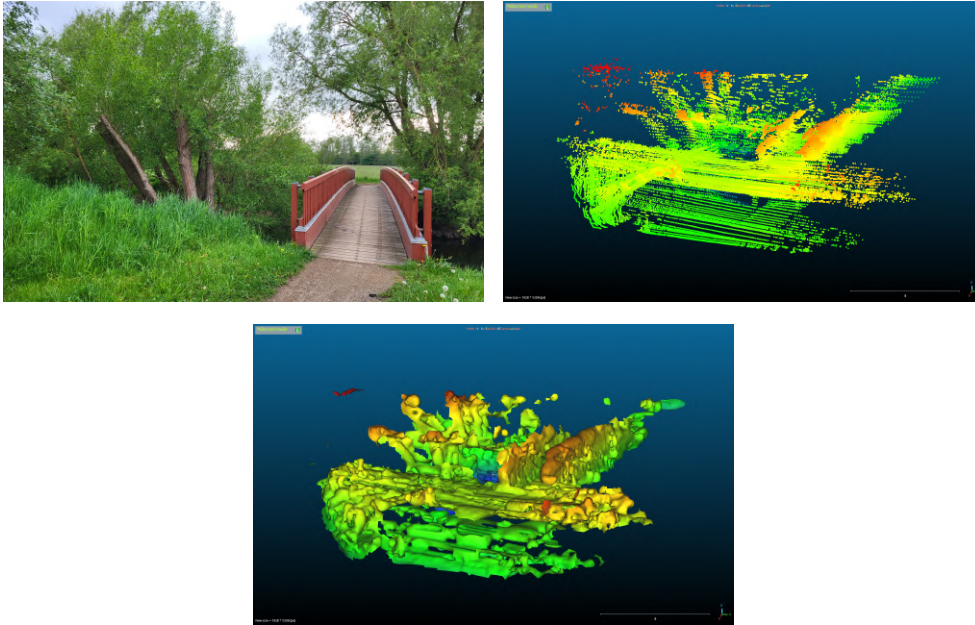


Figure 5.14: Riverside bridge scans (GNSS positioning).

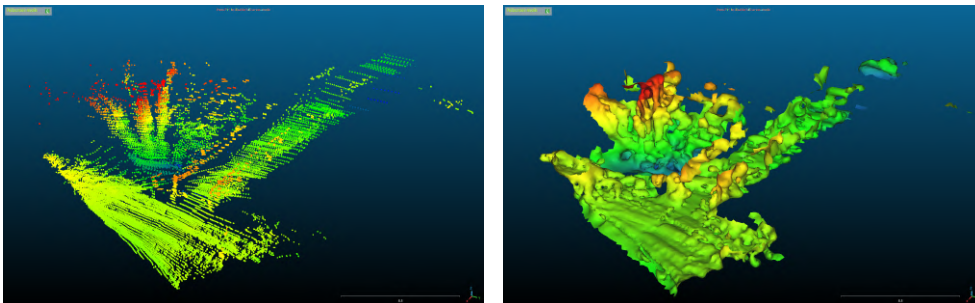


Figure 5.15: Riverside bridge scans (optical flow positioning).

A similar test was performed in a riverside area with a small wooden bridge located there. The optical flow positioning resulted in less noise in the Z axis, but both of the scans are not perfect. The point cloud was obtained in the same way as in the field scans — by a horizontal flight.

5.1.4 Summary

As can be seen in the resulting point clouds, the proposed system lacks the precision for the high-accuracy mapping. In almost all cases, only the general geometry of the scanned objects is registered correctly, while their details are completely lost in the

noise. The quality of the point clouds depends heavily on the positioning accuracy, the scanning pattern, and the drone's speed. The slower the drone travels, the denser the point clouds are.

The scans obtained using the OptiTrack system are far more precise than the ones with the GNSS or the optical flow sensor. But even with this high-precision positioning, some artefacts occur. They may be caused by the drone still travelling too fast or by the inaccuracies in the scan deskewing process.

The ICP correction method seems to help, but only to some extent. The fake 3D LiDAR scan stacking approach is not ideal and can introduce errors. For example, if the skewed part of the scan lays within the buffered scans, the ICP will not be able to correct it perfectly. The ICP method requires scans to have some common features visible, therefore the buffering method can accumulate errors over time. Additionally, it is sensitive to the quality of the initial scans, and its parameters have to be tweaked manually, depending on the particular point cloud. An initial, manual point cloud filtration could potentially further increase the quality of the resulting maps.

5.2 Navigation

Even though the obtained point clouds have too low quality to be used effectively in the precise mapping tasks, they still can prove useful for the navigation. Most often, some safety margin is required to keep a drone away from obstacles, so the low resolution of the OctoMap filtering is not a problem there. What is more, the OctoMap produces sparser point clouds, which are ideal for real-time systems with limited computational resources.

The EGO-Swarm planner performance was tested only in the simulation, as an accidental crash of the real drone could introduce severe delays in the project. Inside the simulation, it was also easy to experiment with the navigation across various environments. However, the communication between the planner and the actual drone was successfully tested in an open, obstacle-free area as well.

The test setup is presented in Figure 5.16 (using the height colour channel). The drone had to travel from the (0 m, 0 m, 3 m) point to arbitrarily chosen (1 m, 13 m, 4 m) point (following the (X, Y, Z) convention). Between these points, there was a gazebo present, and the drone had to avoid it in order to successfully reach its target position. The gazebo was chosen because of its geometry — the drone could pass it in multiple ways. It could fly above, around, or through this construction, and the planner was responsible for finding the appropriate path. The average time that the planner node needed to travel this distance without any obstacles was 55.962 s. The experiment results are presented in Table 5.3.

The drone performs a yaw spin before generating an initial trajectory to gather additional information about the environment. This manoeuvre is necessary as the LiDAR sensor is fixed to the frame and its initial scan is only in a single 2D plane. The results of the test are consistent, and the navigation was successful in most cases. The

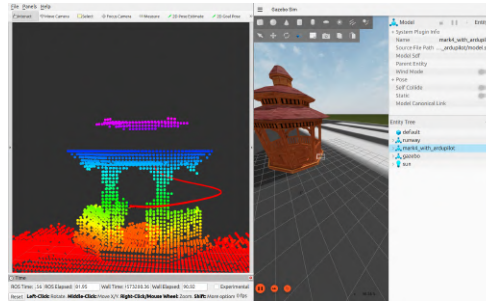


Figure 5.16: Navigation test setup.

No.	Elapsed Time [s]	Number of Emergency Stops	Success
1.	83.572	1	Yes
2.	83.976	1	Yes
3.	64.515	0	Yes
4.	80.288	1	Yes
5.	N/A	2	No (Crash)
6.	81.071	1	Yes
7.	85.604	1	Yes
8.	101.357	2	Yes
9.	N/A	1	No (Stuck)
10.	82.236	1	Yes

Table 5.3: EGO-Swarm planner test results.

differences in the elapsed time come from the planner generating different trajectories in every attempt.

However, some attempts were not successful in reaching the desired target point. One of them ended in the drone crashing into the gazebo's ceiling. This happened, because it flew straight up inside the gazebo. The LiDAR is tilted by 30° , so it creates blind spots directly above and below the drone (see Figure 5.17). Therefore, the planner recognized the blind spot as an unoccupied space and tried to generate the trajectory through it. This configuration also means that the drone is blind for floating obstacles appearing in front of and above it in yet unexplored parts of the space.

Another type of the unsuccessful attempt is the drone being stuck in a suddenly inflated obstacle (see Figure 5.18). After an emergency stop, a yaw spin is performed to safely find a new suitable path. Sometimes during this manoeuvre, a newly discovered obstacle can be so close that after being inflated, the drone finds itself inside the collision area. This blocks the trajectory planner, as it is unable to find the collision-free path from the drone's current position to the target setpoint. To mitigate this problem, the safe distance parameter of the EGO-Swarm planner was increased. Additionally, when an emergency stop occurs, the inflated obstacle dimensions are decreased to 75% of their original values. After the drone finds a new suitable

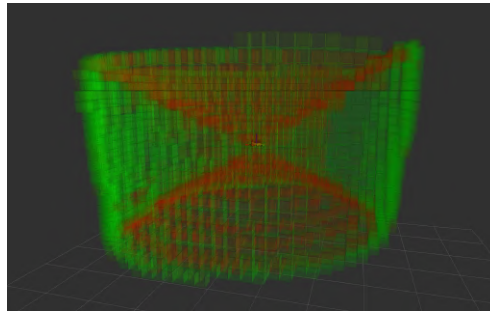


Figure 5.17: Yaw spin blind spot.

trajectory and travels its first 1.5 m, the obstacles are inflated back to their previous volumes. These actions help with escaping from the expanded areas, but do not eliminate this problem completely, as the drone can still be stuck in the already deflated obstacles on rare occasions.

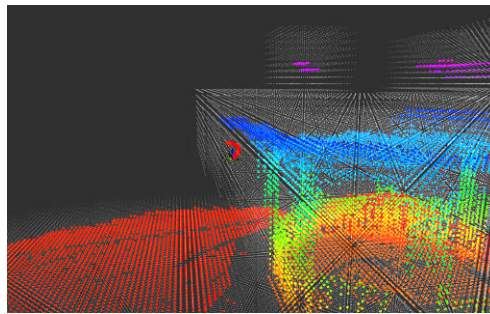


Figure 5.18: Drone stuck in inflated obstacles.

The planner also has problems in semi-enclosed spaces (see Figure 5.19). As the drone's map is being updated continuously, the drone can fly into a room, with an exit, being its entrance in the same time. That would require the drone to go back and look for a different path. However, because of the restricted planning horizon, the planner is not able to find a suitable trajectory around it, effectively blocking the drone in one place.

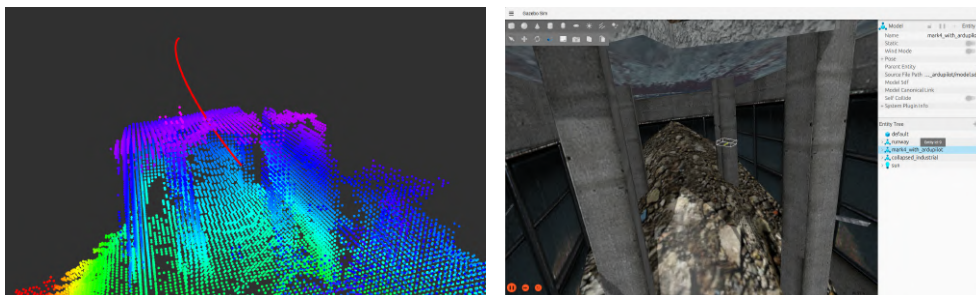


Figure 5.19: Planner stuck in semi-enclosed space.

The EGO-Planner provides a feedforward control, so it does not track the drone's current position. The drone only follows a set of waypoints which should be achieved within a given time with a given speed. Therefore, if its speed is disrupted (for example, by the wind), the trajectory following will desynchronize, and the drone may start moving towards the next waypoint without actually reaching the previous one. This may cause unnecessary emergency stops or even crashes.

Nevertheless, the described problems do not occur often, and if they actually do, it is in cluttered or complex environments with tight unoccupied areas. In the general cases, like flying between buildings or trees, the results are satisfactory, even if the drone does not travel with higher speeds (see Figure 5.20). It is also worth noting that, as the drone explores a new environment, the internal map grows and the planner can generate trajectories with a lower chance of triggering the emergency stops.

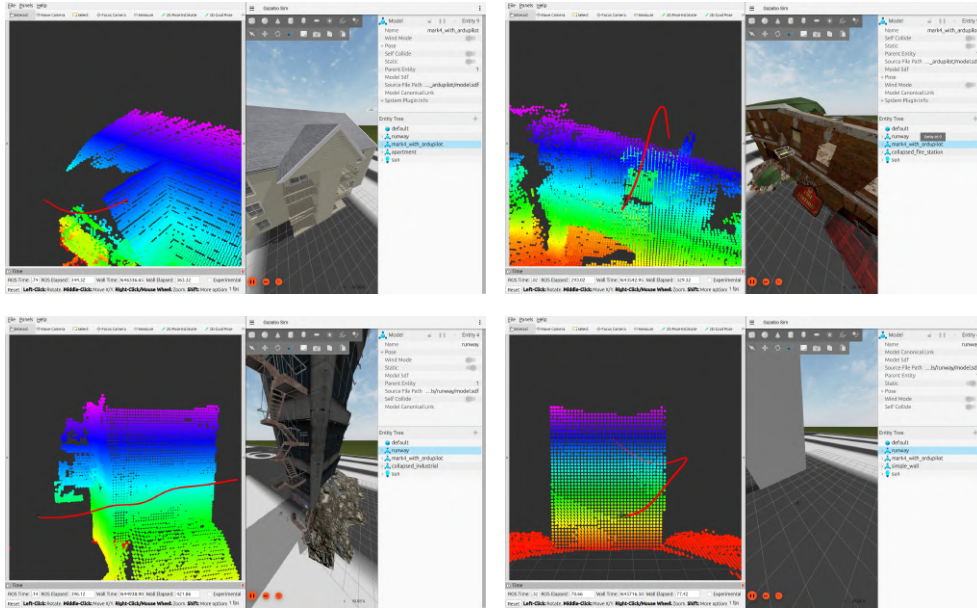


Figure 5.20: Navigation through different environments.

5.3 Cost Estimate

One of the main goals in this project is to keep the cost of the proposed system low. The total cost estimation is shown in Table 5.4. Overall, it can be compared with a price of a standard, pre-built FPV drone (adjusted to a manufacturer's margin). Therefore, the resulting system can be qualified as a low-cost one.

Component	Quantity	Overall Price [€]
JJJIEFPV H743 flight controller 70 A stack	1	~41
GEPRC Mark4 drone kit	1	~65
CNHL 2200 mAh 5S 18.5 V 40C Li-Po battery	1	~23
LDROBOT D500 LiDAR kit	1	~60
Orange Pi Zero 2W with heat sinks	1	~32
SoloGood M10-180C GNSS module	1	~10
MicoAir MTF-02P optical flow sensor	1	~17
SpeedyBee Nano 2.4 GHz ELRS RX	1	~8.5
USB-C and other internal cables	1	~5
FTDI UART-USB converter	1	~2
PLA filament	0.25 kg	~7
Jumper T-Pro S 2.4 GHz ELRS radio controller	1	~116
Total Price [€]		~386.5

Table 5.4: System's cost estimation.

For comparison, the industrial-grade DJI Zenmuse L2 LiDAR Camera [101] costs approximately €12 252 for the drone attachment alone. Another, lower-end 3D LiDAR sensor is the DFRobot CS20 [102] with a price of around €198. These systems promise much better accuracies, but their financial cost is significantly higher as well.

6 Conclusions

In this project, the author explored the applications and limitations of an aerial drone equipped with a low-cost 2D LiDAR sensor. The proposed system was evaluated with different positioning sensors in various environments and scenarios. Filtration methods were proposed to mitigate the influence of the encountered inaccuracies.

The system was proved to have a low accuracy in the mapping tasks, despite using the ICP correction. The quality of the obtained scans is mixed and seems to depend mainly on the positioning accuracy and the flight speed. The best point clouds were obtained with the OptiTrack system, although they still are not at an acceptable quality level. The exact shape of the scanned objects is incomplete in some parts or lost in the noise. Other tested positioning systems are prone to more severe inaccuracies. The GNSS module introduces a drift over time, especially close to the scanned buildings. The optical flow sensor, while having a comparable accuracy, is sensitive to lighting conditions and moving objects. In terms of the altitude estimation, the rangefinder seems to be more accurate than the barometer, but it can be used only in environments with a flat floor.

The proposed system produces rather sparse point clouds. Their density can be increased in expense of lowering the speed of the drone, which is acceptable in the mapping tasks. This way, more details can be captured and preserved.

The described errors cause the ghost walls effect, which the ICP algorithm tried to correct. In most cases, the double surfaces were stitched into a single one, but not always with all the features overlapping correctly. The correction threshold had to be tuned manually depending on a given point cloud. Too large values caused the double walls to be divided into 2 separate objects, while too small ones resulted in barely any correction. Moreover, because of the scan buffering mechanism, if the error is introduced in the middle of the buffered scans, this algorithm will have problems with correcting it.

Despite the poor performance in high-resolution mapping, the system shows promising results in obstacle avoidance navigation. The OctoMap filtering is adaptive for changes in explored spaces, computationally lightweight, and capable of correcting the errors over time, when multiple scans of the same feature are available. The low-resolution of the gathered point clouds is not a problem, because of much larger safety margins around detected obstacles.

The EGO-Swarm trajectory planner was modified to accept the filtered point clouds as inputs. The initial and emergency stop yaw spins were added to feed the planner with additional information about yet unexplored areas. The emergency stop escape mechanism was also enhanced by making inflated obstacles smaller during trajectory replanning. Thanks to that, the risk of the drone being suddenly stuck in an inflated obstacle was significantly lowered.

The planner's performance was evaluated in various simulated environments. It was not successful every time, occasionally failing to find a correct path in cluttered spaces or crashing into obstacles located in the sensor's blind spots. Nevertheless, in general cases with large obstacles (like buildings or trees) and limited speeds, the drone was capable of exploring and navigating its surroundings.

To summarize, the proposed system, despite not being suitable for high-precision mapping tasks, is capable of navigating the drone in typical, low-complexity environments. Its lightweight computational structure means it can be deployed entirely on the companion computer in real time, without the need for a more powerful ground station — all while keeping the overall price relatively low.

6.1 Future Work

In the future, the scope of the current work can be expanded with additional tests. The outdoor scans can be repeated with an RTK system, which would increase their accuracy. Combined with even lower drone speeds and consistent scan paths, it would test the very limits of the current design.

After the ICP correction step, the obtained maps can be further, manually cleaned from obvious outliers. A final, global filtration step could also be beneficial for the overall mapping quality.

Furthermore, the performance achieved by a single fixed 2D LiDAR sensor can be compared with more expensive solutions, like a low-end 3D LiDAR or multiple rotating 2D LiDARs. The mapping process can also be enhanced by adding an RGB camera and matching point cloud and image features to perform a sensor fusion. A proper calibration method could be introduced to ensure the elimination of the mounting inaccuracies. In more extensive tests, a swarm of LiDAR-equipped drones could map an object / environment in much shorter time or with much denser, overlaying point clouds.

In terms of navigation, other trajectory planners can be compared with the currently used EGO-Swarm. As mentioned above, their performance can also be evaluated when using different LiDAR types or even RGB / depth cameras. The currently utilized planner could be further enhanced, for example, by changing the maximal allowed speed based on the occupancy information of the drone's current position in the OctoMap. Additionally, the positioning correction could be further improved by implementing the loop-closure feature, known from typical SLAM systems.

A Appendix

The code used in this project can be found in the following GitHub repositories:

- [lidar_mapper](#) [103],
- [lidar_mapper_visualizer](#) [104].

The videos presenting the results of this project can be found on YouTube:

- [2D LiDAR mapper field scan](#) [105],
- [2D LiDAR mapper OptiTrack test](#) [106],
- [2D LiDAR EGO-Swarm planner navigation](#) [107].

The LaTeX source code for this thesis paper can be found in the following GitHub repository:

- [masters_thesis](#) [108].

Generative AI was used in this project to help with the grammar checking, literature review, solution research, and programming troubleshooting. The author reviewed, edited, and takes full responsibility for the final work.

Bibliography

- [1] OptiTrack. ‘OptiTrack.’ (2026), [Online]. Available: <https://www.optitrack.com/applications/robotics> (visited on 22/02/2026).
- [2] SDU. ‘SDU UAS Center.’ (2026), [Online]. Available: <https://www.sdu.dk/en/forskning/sduuascenter> (visited on 22/02/2026).
- [3] Open Robotics. ‘ROS — Robot Operating System.’ (2025), [Online]. Available: <https://www.ros.org> (visited on 19/04/2026).
- [4] Waveshare. ‘LDROBOT D500.’ (2023), [Online]. Available: https://www.waveshare.com/wiki/D500_LiDAR_Kit (visited on 22/02/2026).
- [5] GEPRC. ‘GEPRC Mark4.’ (2026), [Online]. Available: <https://geprc.com/product/gep-mark4-frame/> (visited on 19/04/2026).
- [6] ArduPilot Dev Team. ‘ArduCopter.’ (2024), [Online]. Available: <https://ardupilot.org/copter/index.html> (visited on 22/02/2026).
- [7] ROTORAMA. ‘Jijie H743.’ (2026), [Online]. Available: <https://www.rotorاما.com/product/jijie-h7-70a-12s> (visited on 22/02/2026).
- [8] Orange Pi. ‘Orange Pi Zero 2W.’ (2026), [Online]. Available: <http://www.orangepi.org/html/hardWare/computerAndMicrocontrollers/details/Orange-Pi-Zero-2W.html> (visited on 22/02/2026).
- [9] A. Filgueira, H. González-Jorge, S. Lagüela, L. Díaz-Vilariño and P. Arias, ‘Quantifying the influence of rain in LiDAR performance,’ *Measurement*, vol. 95, pp. 143–148, 2017, issn: 0263-2241. doi: <https://doi.org/10.1016/j.measurement.2016.10.009>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0263224116305577>.
- [10] A. Wang, C. Li, Y. Liu, Y. Zhuang, C. Bu and J. Xiao, ‘Laser-Based On-line Sliding-Window Approach for UAV Loop-Closure Detection in Urban Environments,’ *International Journal of Advanced Robotic Systems*, vol. 13, no. 2, p. 61, 2016. doi: [10.5772/62755](https://doi.org/10.5772/62755). eprint: <https://doi.org/10.5772/62755>. [Online]. Available: <https://doi.org/10.5772/62755>.
- [11] S. Thrun, M. Diel and D. Hähnel, ‘Scan Alignment and 3-D Surface Modeling with a Helicopter Platform,’ in *Field and Service Robotics: Recent Advances in Reserch and Applications*, S. Yuta, H. Asama, E. Prassler, T. Tsubouchi and S. Thrun, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 287–297, isbn: 978-3-540-32854-4. doi: [10.1007/10991459_28](https://doi.org/10.1007/10991459_28). [Online]. Available: https://doi.org/10.1007/10991459_28.

- [12] S. Bi, C. Yuan, C. Liu, J. Cheng, W. Wang and Y. Cai, ‘A Survey of Low-Cost 3D Laser Scanning Technology,’ *Applied Sciences*, vol. 11, no. 9, 2021, ISSN: 2076-3417. DOI: [10.3390/app11093938](https://doi.org/10.3390/app11093938). [Online]. Available: <https://www.mdpi.com/2076-3417/11/9/3938>.
- [13] G. A. Kumar, A. K. Patil, R. Patil, S. S. Park and Y. H. Chai, ‘A LiDAR and IMU Integrated Indoor Navigation System for UAVs and Its Application in Real-Time Pipeline Classification,’ *Sensors*, vol. 17, no. 6, 2017, ISSN: 1424-8220. DOI: [10.3390/s17061268](https://doi.org/10.3390/s17061268). [Online]. Available: <https://www.mdpi.com/1424-8220/17/6/1268>.
- [14] A. Diosi and L. Kleeman, ‘Laser scan matching in polar coordinates with application to SLAM,’ in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2005, pp. 3317–3322. DOI: [10.1109/IROS.2005.1545181](https://doi.org/10.1109/IROS.2005.1545181).
- [15] X. Hou, Z. Pan, L. Lu *et al.*, ‘LAEA: A 2D LiDAR-Assisted UAV Exploration Algorithm for Unknown Environments,’ *Drones*, vol. 8, no. 4, 2024, ISSN: 2504-446X. DOI: [10.3390/drones8040128](https://doi.org/10.3390/drones8040128). [Online]. Available: <https://www.mdpi.com/2504-446X/8/4/128>.
- [16] B. Zhou, Y. Zhang, X. Chen and S. Shen, ‘FUEL: Fast UAV Exploration Using Incremental Frontier Structure and Hierarchical Planning,’ *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 779–786, 2021. DOI: [10.1109/LRA.2021.3051563](https://doi.org/10.1109/LRA.2021.3051563).
- [17] Y. Zhao, L. Yan, H. Xie, J. Dai and P. Wei, ‘Autonomous Exploration Method for Fast Unknown Environment Mapping by Using UAV Equipped With Limited FOV Sensor,’ *IEEE Transactions on Industrial Electronics*, vol. 71, no. 5, pp. 4933–4943, 2024. DOI: [10.1109/TIE.2023.3285921](https://doi.org/10.1109/TIE.2023.3285921).
- [18] M. Faria, A. S. Ferreira, H. Pérez-Leon, I. Maza and A. Viguria, ‘Autonomous 3D Exploration of Large Structures Using an UAV Equipped with a 2D LIDAR,’ *Sensors*, vol. 19, no. 22, 2019, ISSN: 1424-8220. DOI: [10.3390/s19224849](https://doi.org/10.3390/s19224849). [Online]. Available: <https://www.mdpi.com/1424-8220/19/22/4849>.
- [19] B. Yamauchi, ‘A frontier-based approach for autonomous exploration,’ in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA’97. ‘Towards New Computational Principles for Robotics and Automation’*, 1997, pp. 146–151. DOI: [10.1109/CIRA.1997.613851](https://doi.org/10.1109/CIRA.1997.613851).
- [20] A. Nash, S. Koenig and C. Tovey, ‘Lazy Theta*: Any-Angle Path Planning and Path Length Analysis in 3D,’ *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, pp. 147–154, Jun. 2010. DOI: [10.1609/aaai.v24i1.7566](https://doi.org/10.1609/aaai.v24i1.7566). [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/7566>.

- [21] J. Zhang and S. Singh, ‘LOAM : Lidar Odometry and Mapping in real-time,’ *Robotics: Science and Systems Conference (RSS)*, pp. 109–111, Jan. 2014.
- [22] C. Wen, J. Tan, F. Li *et al.*, ‘Cooperative indoor 3D mapping and modeling using LiDAR data,’ *Information Sciences*, vol. 574, pp. 192–209, 2021, ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2021.06.006>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025521005934>.
- [23] W. Zhen and S. Scherer, *A Unified 3D Mapping Framework using a 3D or 2D LiDAR*, 2018. arXiv: [1810.12515](https://arxiv.org/abs/1810.12515) [cs.RO]. [Online]. Available: <https://arxiv.org/abs/1810.12515>.
- [24] W. Hess, D. Kohler, H. Rapp and D. Andor, ‘Real-Time Loop Closure in 2D LIDAR SLAM,’ in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1271–1278.
- [25] M. Ul Hassan, D. Das and J. Miura, ‘3D Mapping for a Large Crane Using Rotating 2D-Lidar and IMU Attached to the Crane Boom,’ *IEEE Access*, vol. 11, pp. 21 104–21 116, 2023. DOI: [10.1109/ACCESS.2023.3250248](https://doi.org/10.1109/ACCESS.2023.3250248).
- [26] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige and W. Burgard, ‘G2o: A general framework for graph optimization,’ in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3607–3613. DOI: [10.1109/ICRA.2011.5979949](https://doi.org/10.1109/ICRA.2011.5979949).
- [27] T. Liu, M. Carlberg, G. Chen, J. Chen, J. Kua and A. Zakhori, ‘Indoor localization and visualization using a human-operated backpack system,’ in *2010 International Conference on Indoor Positioning and Indoor Navigation*, 2010, pp. 1–10. DOI: [10.1109/IPIN.2010.5646820](https://doi.org/10.1109/IPIN.2010.5646820).
- [28] X. Li, X. Li, G. Hu, Q. Niu and L. Xu, ‘A Low-Cost 3D Mapping System for Indoor Scenes Based on 2D LiDAR and Monocular Cameras,’ *Remote Sensing*, vol. 16, no. 24, 2024, ISSN: 2072-4292. DOI: [10.3390/rs16244712](https://doi.org/10.3390/rs16244712). [Online]. Available: <https://www.mdpi.com/2072-4292/16/24/4712>.
- [29] SLAMTEC. ‘RPLIDAR A1.’ (2026), [Online]. Available: <https://www.slamtec.com/en/lidar/a1> (visited on 19/04/2026).
- [30] SLAMTEC. ‘RPLIDAR C1.’ (2026), [Online]. Available: <https://www.slamtec.com/en/c1> (visited on 19/04/2026).
- [31] SensorLidar. ‘LDROBOT D800.’ (2025), [Online]. Available: <https://www.sensorlidar.com/products/ldrobot-d800-lidar-kit-25meters-dtof-ros-car-slam-navigation-scanning-laser-radar-sensor-support-ros1-and-ros2-for-outdoor> (visited on 22/02/2026).
- [32] ldrobotSensorTeam. ‘ldlidar_stl_ros2.’ (2023), [Online]. Available: https://github.com/ldrobotSensorTeam/ldlidar_stl_ros2 (visited on 19/04/2026).
- [33] ldrobotSensorTeam. ‘ldlidar_stl_ros2.’ (2022), [Online]. Available: https://github.com/ldrobotSensorTeam/ldlidar_stl_sdk (visited on 26/05/2026).

- [34] Waveshare. 'LDROBOT D500 schematics.' (2023), [Online]. Available: <https://www.waveshare.com/wiki/File:DTOF-Coordinate3.png> (visited on 19/04/2026).
- [35] Radxa. 'Radxa ZERO 3W.' (2026), [Online]. Available: <https://radxa.com/products/zeros/zero3w/> (visited on 19/04/2026).
- [36] Raspberry Pi. 'Raspberry Pi Zero 2 W.' (2026), [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/> (visited on 19/04/2026).
- [37] Canonical Ubuntu. 'Get Ubuntu Server.' (2026), [Online]. Available: <https://ubuntu.com/download/server/thank-you?version=24.04.4&architecture=amd64&its=true> (visited on 06/05/2026).
- [38] Open Robotics. 'ROS 2 Documentation: Jazzy.' (2026), [Online]. Available: <https://docs.ros.org/en/jazzy/index.html> (visited on 19/04/2026).
- [39] B-Cube.RC. 'B-CUBE 4PCS Uangel A2807 1300KV Brushless Motor for FPV Freestyle 6inch 7inch Long Range LR7 Drones DIY Parts Multicopter Spare.' (2026), [Online]. Available: <https://bcuberc.com/products/rctosky-4pcs-uangel-a2807-1300kv-brushless-motor-for-fpv-freestyle-6inch-7inch-long-range-lr7-drones-diy-parts-multicopter-spare?variant=46291573440760> (visited on 19/04/2026).
- [40] ROTORAMA. 'Omnibus F4 V3 Plus.' (2026), [Online]. Available: <https://www.rotorاما.com/product/readytosky-f4-v3s-plus> (visited on 22/02/2026).
- [41] iNavFlight. 'inav.' (2026), [Online]. Available: <https://github.com/iNavFlight/inav> (visited on 22/02/2026).
- [42] Betaflight. 'DShot.' (2026), [Online]. Available: <https://betaflight.com/docs/development/API/Dshot> (visited on 19/04/2026).
- [43] MAVLink. 'MAVLink Protocol Overview.' (2026), [Online]. Available: <https://mavlink.io/en/about/overview.html> (visited on 19/04/2026).
- [44] ArduPilot Dev Team. 'Multiwii Serial Protocol (MSP).' (2024), [Online]. Available: <https://ardupilot.org/copter/docs/common-msp-overview.html> (visited on 19/04/2026).
- [45] iNavFlight. 'Serial.' (2025), [Online]. Available: <https://github.com/iNavFlight/inav/blob/master/docs/Serial.md> (visited on 20/04/2026).
- [46] STMicroelectronics. 'STM32H743/753.' (2026), [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32h743-753.html> (visited on 20/04/2026).
- [47] TDK InvenSense. 'ICM-42688-P Datasheet.' (2021), [Online]. Available: https://product.tdk.com/system/files/dam/doc/product/sensor/motion-inertial/imu/data_sheet/ds-000347-icm-42688-p-v1.6.pdf (visited on 20/04/2026).

- [48] SCRIBD. ‘SPL06-001 Preliminary Datasheet.’ (2015), [Online]. Available: <https://www.scribd.com/document/484330234/SPL06-001-datasheet-V2-0> (visited on 20/04/2026).
- [49] ArduPilot Dev Team. ‘DAKEFPV H743 Pro Wiring Diagram.’ (2024), [Online]. Available: https://ardupilot.org/copter/_images/DAKEFPVH743Pro_WiringDiagramTop.png (visited on 19/04/2026).
- [50] MicoAir. ‘MTF-02P.’ (2026), [Online]. Available: https://micoair.com/optical_range_sensor_mtf-02p/ (visited on 20/04/2026).
- [51] SoloGood. ‘SoloGood M10-180C GPS.’ (2026), [Online]. Available: <https://sologood.net/products/sologood-m10gps-m10-180c-m10-180-bds-with-compass-10thgeneration-ublox-for-rc-racing-fpv-drone-airplane-helicopter-quadcopter> (visited on 22/02/2026).
- [52] qst. ‘3-Axis Magnetic Sensor QMC5883L.’ (2016), [Online]. Available: <https://www.tinytronics.nl/image/data/Producten/Sensoren/Magnetisch%20veld/Datasheet-QMC5883L-1.0%20.pdf> (visited on 20/04/2026).
- [53] OptiTrack. ‘12.7 mm (1/2) M4 Markers.’ (2026), [Online]. Available: <https://optitrack.com/accessories/127-mm-12-m4-markers> (visited on 22/04/2026).
- [54] ExpressLRS. ‘High Performance Open Source Radio Control Link.’ (2025), [Online]. Available: <https://www.expresslrs.org> (visited on 25/04/2026).
- [55] SpeedyBee. ‘SpeedyBee Nano 2.4G 2.4G-TCXO 915M ExpressLRS ELRS Receiver.’ (2026), [Online]. Available: <https://www.speedybee.com/speedybee-nano-2-4g-2-4g-tcxo-915m-expresslrs-elrs-receiver/> (visited on 20/04/2026).
- [56] ROTORAMA. ‘Jumper T-Pro S ELRS.’ (2026), [Online]. Available: <https://www.rotorاما.com/product/jumper-t-pro-s-elrs> (visited on 11/05/2026).
- [57] ALLDATASHEET.COM. ‘FT232RL Datasheet (PDF) - Future Technology Devices International Ltd.’ (2005), [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/pdf/144591/FTDI/FT232RL.html> (visited on 22/04/2026).
- [58] Worldsemi. ‘WS2812B.’ (2026), [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf> (visited on 22/04/2026).
- [59] Open Robotics. ‘GAZEBO.’ (2026), [Online]. Available: <https://gazebo.org/home> (visited on 27/04/2026).
- [60] Open Robotics. ‘Getting Started with Gazebo.’ (2026), [Online]. Available: <https://gazebo.org/docs/harmonic/getstarted/> (visited on 01/05/2026).
- [61] ArduPilot Dev Team. ‘Using SITL with Gazebo.’ (2024), [Online]. Available: <https://ardupilot.org/dev/docs/sitl-with-gazebo.html> (visited on 01/05/2026).
- [62] Open Source Robotics Foundation. ‘SDFormat Describe your world.’ (2020), [Online]. Available: <https://sdformat.org> (visited on 01/05/2026).
- [63] Design and Learning Research Group. ‘ACDC4Robot.’ (2024), [Online]. Available: <https://apps.autodesk.com/FUSION/ir/Detail/Index?id=5028052292896011577&appLang=en&os=Win64> (visited on 01/05/2026).

- [64] Open Robotics. ‘XML Robot Description Format (URDF).’ (2023), [Online]. Available: <https://wiki.ros.org/urdf/XML/model> (visited on 01/05/2026).
- [65] Open Robotics. ‘Sensors.’ (2026), [Online]. Available: <https://gazebo.sim.org/docs/harmonic/sensors/> (visited on 01/05/2026).
- [66] Open Robotics. ‘Gazebo Fuel.’ (2026), [Online]. Available: <https://app.gazebo.sim.org/dashboard> (visited on 03/05/2026).
- [67] SDU UAS Center. ‘UASwork.’ (2020), [Online]. Available: https://gitlab.com/uaswork/sdu-uas-center/eit-playground/-/tree/master/models/optitrack_prime?ref_type=heads (visited on 03/05/2026).
- [68] Willow Garage, Inc. ‘rviz2.’ (2012), [Online]. Available: <https://docs.ros.org/en/rolling/p/rviz2/> (visited on 01/05/2026).
- [69] Myzhar. ‘ldrobot-lidar-ros2.’ (2026), [Online]. Available: <https://github.com/Myzhar/ldrobot-lidar-ros2/> (visited on 29/05/2026).
- [70] Open Robotics. ‘sensor_msgs/LaserScan Message.’ (2019), [Online]. Available: https://docs.ros.org/en/lunar/api/sensor_msgs/html/msg/LaserScan.html (visited on 04/05/2026).
- [71] Open Robotics. ‘Tf2.’ (2026), [Online]. Available: <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Tf2.html> (visited on 05/05/2026).
- [72] ArduPilot Dev Team. ‘Extended Kalman Filter Navigation Overview and Tuning.’ (2024), [Online]. Available: <https://ardupilot.org/dev/docs/extended-kalman-filter.html#extended-kalman-filter> (visited on 22/02/2026).
- [73] V. Ermakov. ‘local_position.cpp.’ (2017), [Online]. Available: http://docs.ros.org/en/jade/api/mavros/html/local__position_8cpp_source.html (visited on 06/05/2026).
- [74] mavlink. ‘MAVROS.’ (2026), [Online]. Available: <https://github.com/mavlink/mavros> (visited on 06/05/2026).
- [75] S. Ma, Z. Liu, C. Wang *et al.*, ‘Ghost imaging LiDAR via sparsity constraints using push-broom scanning,’ *Opt. Express*, vol. 27, no. 9, pp. 13 219–13 228, Apr. 2019. doi: 10.1364/OE.27.013219. [Online]. Available: <https://opg.optica.org/oe/abstract.cfm?URI=oe-27-9-13219>.
- [76] Z. Wang, H. Li, S. Wang, L. Song and J. Chen, ‘Methodology and Modeling of UAV Push-Broom Hyperspectral BRDF Observation Considering Illumination Correction,’ *Remote Sensing*, vol. 16, no. 3, 2024, ISSN: 2072-4292. doi: 10.3390/rs16030543. [Online]. Available: <https://www.mdpi.com/2072-4292/16/3/543>.
- [77] Livox. ‘Livox Open Source Sharing: About LiDAR Distortion Removal.’ (2026), [Online]. Available: <https://www.livoxtech.com/showcase/211220> (visited on 06/05/2026).
- [78] The <laser_geometry> contributors. ‘laser_geometry.’ (2024), [Online]. Available: https://docs.ros.org/en/jazzy/p/laser_geometry/ (visited on 07/05/2026).

- [79] Open Robotics. ‘sensor_msgs/PointCloud2 Message.’ (2026), [Online]. Available: https://docs.ros.org/en/noetic/api/sensor_msgs/html/msg/PointCloud2.html (visited on 07/05/2026).
- [80] AudioSceneDescriptionFormat. ‘Spherical Linear Interpolation (Slerp).’ (2024), [Online]. Available: <https://splines.readthedocs.io/en/latest/rotation/slerp.html> (visited on 07/05/2026).
- [81] E. Marder-Eppstein and W. Meeussen. ‘tf2_ros::MessageFilter< M > Class Template Reference.’ (2017), [Online]. Available: https://docs.ros.org/en/jade/api/tf2_ros/html/c++/classtf2__ros_1_1MessageFilter.html (visited on 09/05/2026).
- [82] European Space Agency. ‘SBAS Fundamentals.’ (2018), [Online]. Available: https://gssc.esa.int/navipedia/index.php/SBAS_Fundamentals (visited on 10/05/2026).
- [83] BBC. ‘Sweden accuses Russia of GPS jamming over Baltic Sea.’ (2025), [Online]. Available: <https://www.bbc.com/news/articles/clyx3ly54veo> (visited on 10/05/2026).
- [84] Point Cloud Library. ‘Removing outliers using a StatisticalOutlierRemoval filter.’ (2026), [Online]. Available: https://pcl.readthedocs.io/projects/tutorials/en/latest/statistical_outlier.html (visited on 12/05/2026).
- [85] Point Cloud Library. ‘Removing outliers using a Conditional or RadiusOutlier removal.’ (2026), [Online]. Available: https://pcl.readthedocs.io/projects/tutorials/en/latest/remove_outliers.html#remove-outliers (visited on 12/05/2026).
- [86] Point Cloud Library. ‘Downsampling a PointCloud using a VoxelGrid filter.’ (2026), [Online]. Available: https://pcl.readthedocs.io/projects/tutorials/en/latest/voxel_grid.html#voxelgrid (visited on 12/05/2026).
- [87] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss and W. Burgard, ‘OctoMap: An efficient probabilistic 3D mapping framework based on octrees,’ *Autonomous Robots*, 2013, Software available at <https://octomap.github.io>. DOI: 10.1007/s10514-012-9321-0. [Online]. Available: <https://octomap.github.io>.
- [88] S. Sharma. ‘Octree Algorithms in 3D Design: Managing Complexity with Efficient Spatial Structures.’ (2026), [Online]. Available: <https://beegraphy.com/blog/3d-design-using-octree-algorithms/> (visited on 14/05/2026).
- [89] OctoMap. ‘octomap_mapping.’ (2026), [Online]. Available: https://github.com/OctoMap/octomap_mapping (visited on 14/05/2026).
- [90] P. Besl and N. D. McKay, ‘A method for registration of 3-D shapes,’ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992. DOI: 10.1109/34.121791.

- [91] T. Pozderac, J. Velagic and D. Osmankovic, '3D Mapping Based on Fusion of 2D Laser and IMU Data Acquired by Unmanned Aerial Vehicle,' Apr. 2019, pp. 1533–1538. DOI: [10.1109/CoDIT.2019.8820583](https://doi.org/10.1109/CoDIT.2019.8820583).
- [92] Jaykumaran. 'Understanding Iterative Closest Point (ICP) Algorithm with Code.' (2025), [Online]. Available: <https://learnopencv.com/iterative-closest-point-icp-explained/> (visited on 17/05/2026).
- [93] X. Zhou, J. Zhu, H. Zhou, C. Xu and F. Gao, 'EGO-Swarm: A Fully Autonomous and Decentralized Quadrotor Swarm System in Cluttered Environments,' in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 4101–4107. DOI: [10.1109/ICRA48506.2021.9561902](https://doi.org/10.1109/ICRA48506.2021.9561902).
- [94] X. Zhou, Z. Wang, H. Ye, C. Xu and F. Gao, 'EGO-Planner: An ESDF-Free Gradient-Based Local Planner for Quadrotors,' *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2021. DOI: [10.1109/LRA.2020.3047728](https://doi.org/10.1109/LRA.2020.3047728).
- [95] P. E. Hart, N. J. Nilsson and B. Raphael, 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths,' *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. DOI: [10.1109/TSSC.1968.300136](https://doi.org/10.1109/TSSC.1968.300136).
- [96] ZJU-FAST-Lab. 'ego-planner-swarm.' (2025), [Online]. Available: https://github.com/ZJU-FAST-Lab/ego-planner-swarm/tree/ros2_version (visited on 17/05/2026).
- [97] M. Kazhdan, M. Bolitho and H. Hoppe, 'Poisson surface reconstruction,' in *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, ser. SGP '06, Cagliari, Sardinia, Italy: Eurographics Association, 2006, pp. 61–70, ISBN: 3905673363.
- [98] M. Kazhdan. 'Adaptive Multigrid Solvers (Version 18.76).' (2023), [Online]. Available: <https://www.cs.jhu.edu/~misha/Code/PoissonRecon/Version18.76/> (visited on 20/06/2026).
- [99] CloudCompare Wiki. 'Poisson Surface Reconstruction (plugin).' (2015), [Online]. Available: https://www.cloudcompare.org/doc/wiki/index.php/Poisson_Surface_Reconstruction_plugin (visited on 20/05/2026).
- [100] CloudCompare. 'CloudCompare.' (2026), [Online]. Available: <https://www.cloudcompare.org> (visited on 20/05/2026).
- [101] DJI Store Danmark. 'DJI Zenmuse L2.' (2026), [Online]. Available: https://djistoreddanmark.dk/en/products/dji-zenmuse-l2?_pos=3&_sid=b24bdd23c&_ss=r (visited on 30/05/2026).
- [102] OpenELAB Technology Ltd. 'DFRobot CS20 LiDAR Ranging Sensor — Single-Point Laser Distance Sensor UART I2C.' (2026), [Online]. Available: https://openelab.io/products/dfrobot-cs20-3dtof-lidar?_pos=1&_sid=ffaecead7&_ss=r (visited on 30/05/2026).

- [103] jango175. 'lidar_mapper.' (2026), [Online]. Available: https://github.com/jango175/lidar_mapper (visited on 30/05/2026).
- [104] jango175. 'lidar_mapper_visualizer.' (2026), [Online]. Available: https://github.com/jango175/lidar_mapper_visualizer (visited on 30/05/2026).
- [105] J. Górski. '2D LiDAR mapper field scan.' (2026), [Online]. Available: <https://youtu.be/rYdUt9qeAig> (visited on 21/05/2026).
- [106] J. Górski. '2D LiDAR mapper OptiTrack test.' (2026), [Online]. Available: <https://youtu.be/vxvr4-iBznk> (visited on 21/05/2026).
- [107] J. Górski. '2D LiDAR EGO-Swarm planner navigation.' (2026), [Online]. Available: <https://youtu.be/siKGZ6RW-xc> (visited on 13/05/2026).
- [108] jango175. 'masters_thesis.' (2026), [Online]. Available: https://github.com/jango175/masters_thesis (visited on 30/05/2026).